

PROBLEMI di OTTIMIZZAZIONE

Permette l'ordinamento
delle soluzioni

FUNZIONE OBIETTIVO $c: F \rightarrow \mathbb{R}$ su un INSIEME AMMISSIBILE F

$$(P) \quad \max \{c(x) : x \in F\} = - \min \{-c(x) : x \in F\}$$

SOLUZIONE AMMISSIBILE $x \in F \subset F'$
NON AMMISSIBILE $x \in F' \setminus F$

SOLUZIONE OTTIMA x^+ deve essere

- AMMISSIBILE $x^+ \in F$
- La MIGLIORE $\forall x \in F, c(x^+) \leq c(x)$

NON per forza UNICA

e.g. $\max \left\{ x_1 : \begin{array}{l} 0 \leq x_1 \leq 1 \\ 0 \leq x_2 \leq 1 \end{array} \right\}$

VALORE OTTIMO $z(P) \leftarrow$

Quando NON ESISTE?

- F INFERIORMENTE SUPERIORMENTE ILLIMITATO $z(P) = +\infty$
 $\forall M \in \mathbb{R} \exists x \in F: c(x) \leq M$
- F VUOTO $z(P) = -\infty$

TOLLERANZA Nella pratica abbiamo una tolleranza per la soluzione ottimale

e.g. DOUBLE PRECISION $1e-16$

$$|c(x') - c(x'')| \leq 1e-16 \Rightarrow \text{SOLUZIONE OTTIMA } \min \{|c(x')|, |c(x'')|\}$$

Assegnando valori ai parametri
di un problema, otteniamo un' ISTANZA

→ Noi vogliamo risolvere
PROBLEMI e NON ISTANZE

PROBLEMI di DECISIONE (D) $\exists x \in F$?

Corrispondono a problemi di OTTIMIZZAZIONE
con $c(x)$ COSTANTE

Facilmente collegati a problemi di OTTIMIZZAZIONE: $(P_K) \exists x: x \in F, c(x) \geq K$?
Esiste un modo per guadagnare almeno K ?

PROBLEMI di DECISIONE $\Rightarrow$ PROBLEMI di OTTIMIZZAZIONE
NP-COMPLETI NP-HARD

ALGORITMI ESATTI $A: p \rightarrow x^*$ OTTIMA

EURISTICI $A: p \rightarrow \bar{x} \in F$

FORSE trovo una soluzione che FORSE è ottimale

ϵ -APPROSSIMATO restituisce una soluzione non troppo peggio della ottima

$$z(p) - c(\bar{x}) \leq \epsilon \quad \text{ERRORE ASSOLUTO di } \bar{x}$$

$$\frac{z(p) - c(\bar{x})}{z(p)} \quad \text{RELATIVO}$$

Ma come calcolo $z(p)$? **RILASSAMENTO !**

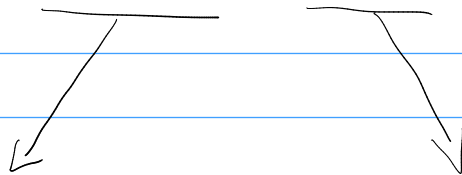
$$(\bar{p}) \max \{ \bar{c}(x) : x \in \bar{F} \}$$

$$\Rightarrow z(\bar{p}) \geq z(p)$$

- Non ha meno soluzioni $\bar{F} \supseteq F$
- $\bar{c}(\bar{x}) \geq c(\bar{x}) \quad \bar{x} \in F$

SE $\epsilon \geq z(\bar{p}) - c(\bar{x}) \geq z(p) - c(\bar{x})$ ALLORA HO SOLUZIONE
 ϵ -APPROSSIMATA

PROGRAMMAZIONE LINEARE INTERA



POSSO AVERE SOLO

EQUAZIONI LINEARI

(no prodotti di var, no Γ , no log, etc...)

COSTANTE * VAR + COST * VAR + ...

↳ e.g. c_i è COSTANTE

VARIABILI che assumono

solo valori INTERI

VARIABILE di SCELTA $x_i \in \{0, 1\} \quad i = 1, \dots, n$

E.g. costo totale degli oggetti rubati $\sum_{i=1}^n c_i x_i$

ALBERO di COPERTURA MINIMO di UN GRAFO

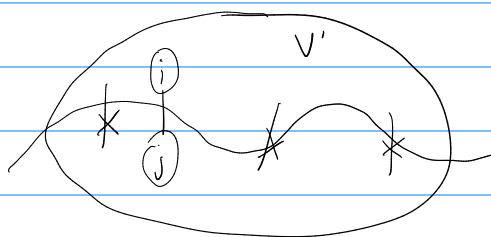
$x_{ij} \in \{0, 1\}$ $\{i, j\} \in E$ $\min \sum_{\{i, j\} \in E} c_{ij} x_{ij}$
Prendo un lato? Nodi Funzione obiettivo

VINCOLO di COPERTURA: non deve succedere che con

un taglio del grafo ci

sia assenza di collegamenti

$V' \subset V, \quad V' \supset \emptyset$



$$\sum_{i \in V', j \notin V'} x_{i,j} \geq 1$$

RELAZIONI LOGICHE

NOT $b = \sim a$ $x_b = 1 - x_a$

AND $c = a \wedge b$
$$\begin{cases} x_c \leq x_a & x_c \leq x_b \\ x_c \geq x_a + x_b - 1 \end{cases}$$

OR $c = a \vee b$
$$\begin{cases} x_c \geq x_a & x_c \geq x_b \\ x_c \leq x_a + x_b \end{cases}$$

$\Rightarrow$ $a \Rightarrow b$ $x_b \geq x_a$

XOR $c = a \oplus b$
$$\begin{cases} x_c \geq x_a - x_b & x_c \geq x_b - x_a \\ x_c \leq x_a + x_b & x_c \leq 2 - x_a - x_b \end{cases}$$

CNF Date un'espressione in CNF possiamo scrivere i vincoli del modello come la somma delle variabili in una clause (negative se negate) maggiore o uguale a 1 meno il numero di clause negate

e.g. $F = (x_1 \vee \sim x_3 \vee \sim x_4) \wedge (x_2 \vee x_3 \vee \sim x_5) \wedge (x_3 \vee \sim x_4 \vee x_5)$

$$\begin{aligned} & x_1 - x_3 - x_4 \geq -1 \\ & \rightarrow x_2 + x_3 - x_5 \geq 0 \\ & x_3 - x_4 + x_5 \geq 0 \end{aligned}$$

VINCOLI

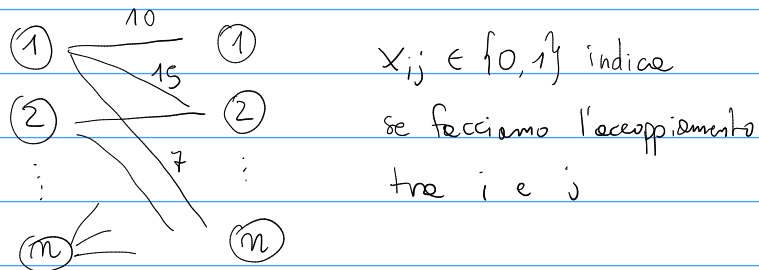
Notiamo che la VERIFICA DELLA SODDISFACIBILITÀ (SAT) corrisponde al problema di ottimizzazione con $\max \emptyset$ e le clause come i vincoli

$\Downarrow$

NP-COMPLETO $\Rightarrow$ PL1 $\in$ NP-COMPLETO

PROBLEMA di ASSEGNAMENTO

Grafo di assegnamento con costi c_{ij}



FUNZIONE OBIETTIVO $\min \sum_{i=1}^m \sum_{j=1}^n c_{ij} x_{ij}$

VINCOLI

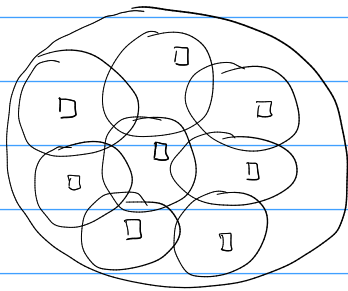
$$\sum_{j=1}^n x_{ij} = 1$$
$$\sum_{i=1}^m x_{ij} = 1$$

SEMI ASSEGNAMENTO
Ogni nodo è accoppiato
con un solo altro nodo
(ASSEGNAMENTO)

PROBLEMA DEL COMMESO VIAGGIATORE

Combino i vincoli del problema di ASSEGNAMENTO con quelli
dell'ALBERO MINIMO di COPERTURA

PROBLEMA DELL'ASSEGNAMENTO DI FREQUENZE



Dato n ANTENNE devo coprire lo spazio con delle FREQUENZE $F = \{f_1, \dots, f_K\}$ senza che si sovrappongono

$x_{hi} \in \{0, 1\}$ che indica se assegno la frequenza h all'antenna i

VINCOLI

$$\sum_{h=1}^K x_{hi} = 1$$

Ogni stazione ha 1 frequenza

$$x_{hi} + x_{hj} \leq 1$$

Una frequenza non è assegnata a stazioni vicine

FUNZIONE

OBIETTIVO

$$y_h \in \{0, 1\}$$

$$y_h = \begin{cases} 1 & \text{se ho usato la frequenza } h \\ 0 & \text{altrimenti} \end{cases}$$

Invento una variabile AUSILIARIA

NB: devo collegarla a una variabile STRUTTURALE

$$x_{hi} \Rightarrow y_h \longrightarrow x_{hi} \leq y_h$$

$$\min \sum_{h=1}^K y_h$$

VARIANTE con GRUPPI di FREQUENZE - SET COVERING

$$F = \{f_1, f_2, \dots, f_K\} \quad S = \{S_1, S_2, \dots, S_m\} \quad S_1 = \{f_1, f_3\}, S_2 = \{f_2, f_6\}, \dots$$

Ad ogni "mezzetto" di frequenze S è assegnato un costo c

$$x_i \in \{0, 1\} \quad \begin{cases} 1 & \text{compro un mezzetto} \\ 0 & \text{altrimenti} \end{cases}$$

VINCOLI

$$\sum_{i: f_h \in S_i} x_i \geq 1$$

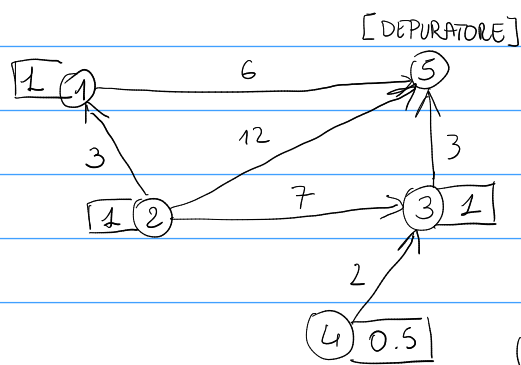
FUNZIONE OBIETTIVO

$$\min \sum_{i=1}^m c_i x_i$$

SET PARTITIONING

=

PROBLEMA del FLUSSO di COSTO MINIMO



FUNZIONE
OGGETTIVO

$x_{ij} \geq 0$ quanto scorre nella condotta

$$\min 6x_{15} + 3x_{21} + 7x_{23} + 12x_{25} + 3x_{35} + 2x_{43}$$

VINCOLI

(Legge di conservazione del Flusso)

$$x_{15} = x_{21} + 1 \rightarrow x_{15} - x_{21} = 1$$

$$x_{21} + x_{23} + x_{25} = 4$$

$$x_{35} - x_{23} - x_{43} = 1$$

$$x_{43} = 0.5$$

$$-x_{15} - x_{25} - x_{35} = -3.5$$

GENERALIZZAZIONE - RETI di FLUSSO

$$G = (N, A) \quad c: A \rightarrow \mathbb{R} \quad c = [c_{ij}]_{(i,j) \in A} \quad \text{COSTO}$$

$$b: N \rightarrow \mathbb{R} \quad b = [b_i]_{i \in N} \quad \text{PRODUZIONE (DEFICIT)}$$

$$u: A \rightarrow \mathbb{R}_+ \quad u = [u_{ij}]_{(i,j) \in A} \quad \text{CAPACITÀ MASSIMA di FLUSSO}$$

$$x = [x_{ij}]_{(i,j) \in A} \quad \text{VETTORE di VARIABILI di FLUSSO}$$

FUNZIONE OGGETTIVO $\min \sum_{(i,j) \in A} c_{ij} x_{ij}$

VINCOLI

$$\sum_{(j,i) \in BS(i)} x_{ji} - \sum_{(i,j) \in FS(i)} x_{ij} = b_i$$

FLUSSO ENTRANTE FLUSSO USCENTE

→ POSITIVO → POZZO
→ NEGATIVO → SORGENTE

$$\sum_{i \in N} b_i = 0 \quad \text{Tutto ciò che è prodotto deve essere consumato}$$

PROBLEMA DEL CAMMINO MINIMO

$$G(N, A) \quad c: A \rightarrow \mathbb{R} \quad s \in N \quad t \in N \quad (s \neq t)$$

GRAFO COSTI PARTENZA DESTINAZIONE

Si può scrivere come caso particolare del flusso di costo minimo

$$b_i = \begin{cases} -1 & i = s \\ 1 & i = t \\ 0 & \text{ALTRIMENTI} \end{cases} \quad u_{ij} = +\infty$$

NB Vogliamo evitare soluzioni frazionarie $\rightarrow x_{ij} \in \{0, 1\}$
 anche se comunque il cammino minimo migliore sarà sempre intero

CAMMINO MINIMO VINCOLATO

Aggiunge le lunghezze $[l_{ij}]_{(i,j) \in A}$ e la lunghezza massima L

Vincoli $\sum_{(i,j) \in A} l_{ij} x_{ij} \leq L$

ESEMPIO: ROUTING

$$G = (N, A) \quad s_m \quad t_m \quad \delta_m \quad m = 1 \dots K \quad u_{ij} \in \mathbb{R}_{++} \quad c_{ij}$$

PARTENZA ARRIVO BANDA PACCHETTI CAPACITÀ LINK COSTO

$$x_{ij}^h = \begin{cases} 1 & \text{se l'arco } (i,j) \\ & \text{appartiene al cammino } h \\ \emptyset & \text{ALTRIMENTI} \end{cases}$$

$$i \in N \quad h = 1 \dots K \quad x_{ij}^h \in \{0, 1\}$$

$$\sum_{(i,j) \in \delta^h(s)} x_{ij}^h - \sum_{(i,j) \in \delta^h(t)} x_{ij}^h = \begin{cases} -1 & i = s^h \\ 1 & i = t^h \\ 0 & \text{ALTRIMENTI} \end{cases}$$

FUNZIONE
OGGETTIVO $\min \sum_{(i,j) \in A} c_{ij} \sum_{h=1}^K x_{ij}^h$

$$\sum_{h=1}^K \delta_{ij}^h x_{ij}^h \leq u_{ij} \quad (\text{VINCOLO BANDA})$$

VARIABILI DISCRETE

- ① Cosa succede se voglio scegliere tra più di due opzioni?

$y_{ij} \in \{a, b, c, \dots\}$ Indica quanto è grosso il "condotto"

↓ DISCRETIZZO

$$y_{ij} = a \cdot z_{ij}^1 + b \cdot z_{ij}^2 + c \cdot z_{ij}^3 + \dots \quad z_{ij}^m \in \{0, 1\}$$

$x_{ij} \leq y_{ij}$ e sostituisco x_{ij} con y_{ij}

- ② La condotta può non essere costruita oppure essere compresa tra a e b

$$y_{ij} \begin{cases} \rightarrow \emptyset \\ \rightarrow [a, b] \end{cases} \Rightarrow y_{ij} \in \{0\} \cup [a, b]$$

$$a \cdot z_{ij} \leq y_{ij} \leq b \cdot z_{ij} \quad z_{ij} \in \{0, 1\}$$

z sceglie se y vale $\emptyset$ o meno

$$③ y_{ij} \in \{0\} \cup [a, b] \cup \{2\} \cup [c, d]$$

⇓ COMBINO ① e ②

$$z_{ij}^1 + z_{ij}^2 + z_{ij}^3 \leq 1 \quad z_{ij} \in \{0, 1\}$$

$$w_{ij}^1 \quad w_{ij}^3$$

⇓

$$y_{ij} = w_{ij}^1 + 2 z_{ij}^2 + w_{ij}^3$$

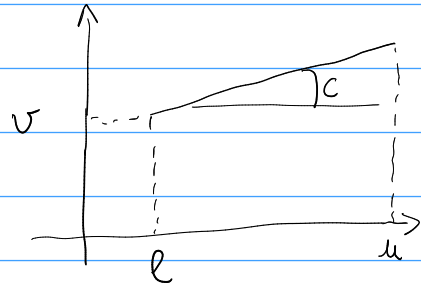
$$a z_{ij}^1 \leq w_{ij}^1 \leq b z_{ij}^1$$

$$c z_{ij}^3 \leq w_{ij}^3 \leq d z_{ij}^3$$

FUNZIONI NON LINEARI

CARICO FISSO

$$f(x) = \begin{cases} 0 & x=0 \\ v+cx & x \in [l, u] \end{cases}$$



Se non produciamo nulla non paghiamo nulla, altrimenti paghiamo un costo fisso v più un costo per quanto produciamo (fino ad un massimo di u)

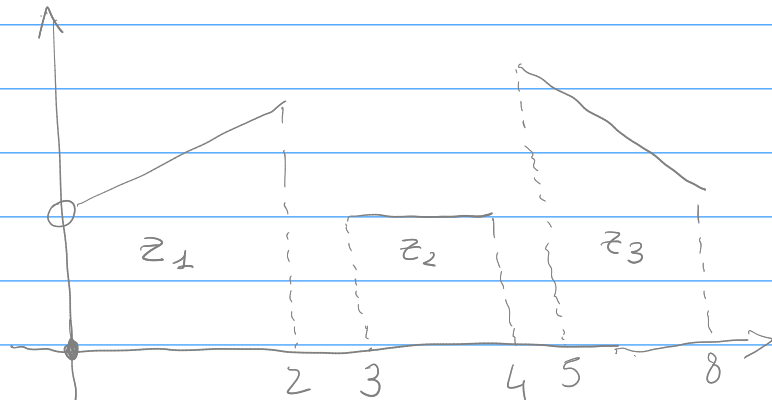
$z \in \{0, 1\}$ FUN. OBIET. $\min v z + c x$ VINCOLI $0 z \leq x \leq u z$

ESEMPIO ORACLE



$$\min \{ f(x) : x \in [0, 8] \}$$

$$f(x) = \begin{cases} 0 & x=0 \\ 1+2x & x \in (0, 2] \\ 3 & x \in [3, 4] \\ 4-3x & x \in [5, 8] \end{cases}$$



VINCOLI

$$z_1 + z_2 + z_3 \leq 1 \quad z^i \in \{0, 1\} \quad i=1..3$$

$$x = w^1 + w^2 + w^3$$

$$0 \leq w^1 \leq 2z^1$$

$$3z^2 \leq w^2 \leq 4z^2$$

$$5z^3 \leq w^3 \leq 8z^3$$

FUN. OBIETTIVO

$$\min z^1 + 2w^1 + 3z^2 + w^2 + 4z^3 - 3w^3$$

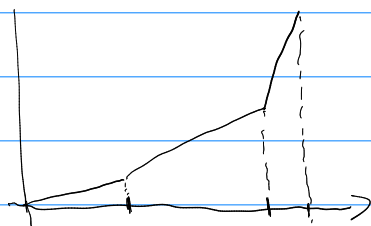
ESEMPIO FONDERIA II

Costo ferro: $[0, 350] \text{ kg}$ 0.03€ $x = x^1 + x^2 + x^3$
 $[350, 750] \text{ kg}$ 0.05€ $z^1 \quad z^2 \quad z^3 \quad z \in \{0, 1\}$
 $[750, 1000] \text{ kg}$ 0.08€

VINCOLI $350 z^2 \leq x^1 \leq 350 z^1 \quad z^2 \leq z^1$ Per comprare nel secondo intervallo
 $400 z^3 \leq x^2 \leq 400 z^2 \quad z^3 \leq z^2$ devo aver terminato il primo
 $0 \leq x^3 \leq 250 z^3$

FUNZIONE OBIETTIVO $0.03 x^1 + 0.05 x^2 + 0.08 x^3$

MA! La funzione è CONNESSA



$0.03 < 0.05 < 0.08$

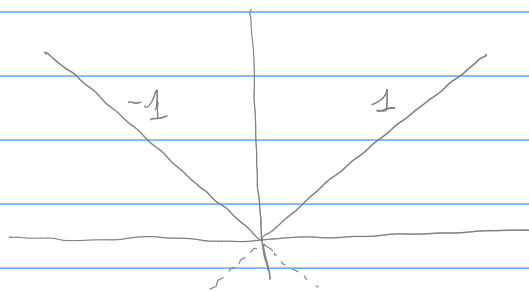
VINCOLI $0 \leq x^1 \leq 350$
 $0 \leq x^2 \leq 400$
 $0 \leq x^3 \leq 250$

Anche se ho soluzioni AMMISSIBILI
dove non "riempio" il primo intervallo
prima di passare agli altri, la CONVESSITÀ
fa in modo che queste non siano mai ottime

Quindi NON SERVONO VARIABILI BINARIE

ESEMPIO VALORE ASSOLUTO

$\min \{ |x| : x \in [2, 7] \}$ Non è INTERA!



La funzione è convessa perché, da Sx a Dx ,
quando la pendenza cambia AUMENTA

FUNZIONE OBIETT. $\min \{ v : v \geq x, v \geq -x \mid x \in [2, 7] \}$

ESEMPIO NORMA INFINITO $\min \{ \|x\|_\infty : x \in X \}$ $\|x\|_\infty = \max \{ |x_i| : i=1 \dots n \}$
diventa $\min \{ v : v \geq x_i, v \geq -x_i \quad i=1 \dots n \}$

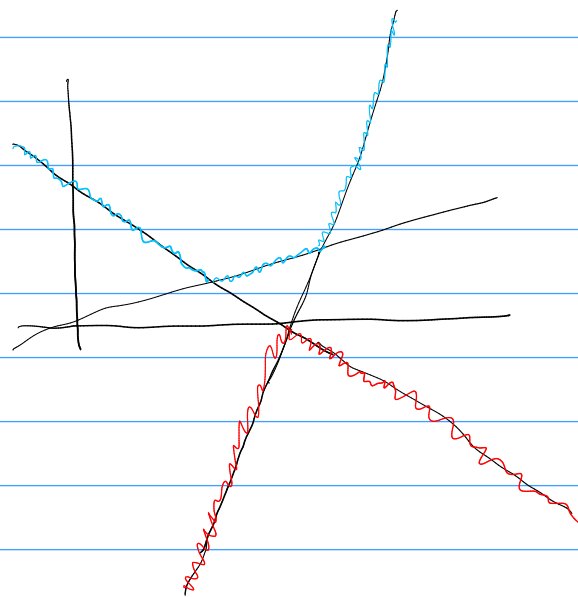
GENERALIZZAZIONE DI FUNZIONI LINEARI

$$f(x) = \boxed{\max} \{ c_1 x + b_1, \dots, c_k x + b_k \}$$

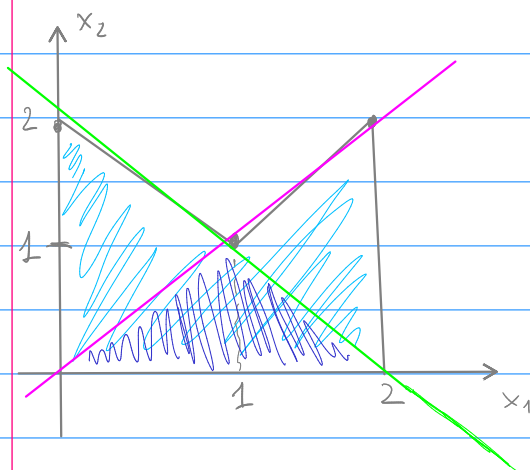
$$\min \{ f(x) \} = \min \{ v : v \geq c_1 x + b_1 \\ v \geq c_k x + b_k \}$$

$$f(x) = \boxed{\min} \{ c_1 x + b_1, \dots, c_k x + b_k \}$$

$$\max \{ f(x) \} = \max \{ v : v \leq c_1 x + b_1 \\ v \leq c_k x + b_k \}$$



ESEMPIO ORACLE



Voglio rappresentare la regione ammissibile

VINCOLI

$$0 \leq x_1 \leq 2$$

$$x_1 + x_2 \leq 2$$

$$0 \leq x_2 \leq 2$$

$$-x_1 + x_2 \leq 0$$

NON VA BENE!

NON si può scrivere come PROGRAMMAZIONE LINEARE perché è NON CONVESSO
quindi usiamo la PROGRAMMAZIONE LINEARE INTERA



VINCOLI DISGIUNTIVI

VINCOLO 1 può essere violato $\left\{ \begin{array}{l} z_1 \end{array} \right.$

VINCOLO 2 può essere violato $\left\{ \begin{array}{l} z_2 \end{array} \right.$

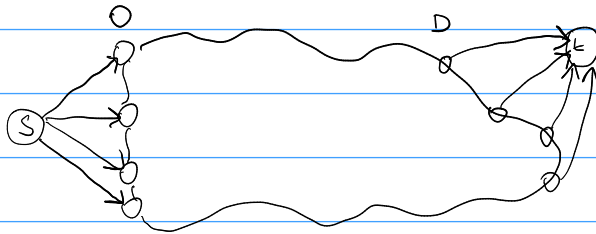
MA NON ENTRAMBI

$$z_1 + z_2 \leq 1 \quad z_1, z_2 \in \{0, 1\}$$

TRASFORMAZIONI EQUIVALENTI

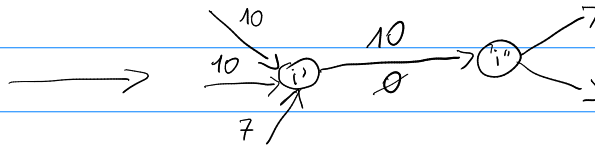
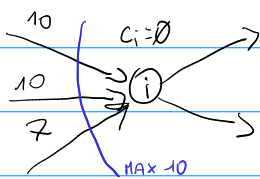
SINGOLA SORGENTE / POZZO

Prendo tutti i nodi sorgente O e pozzi D e li collego rispettivamente a una SUPERSORGENTE S e un SUPERPOZZO t .



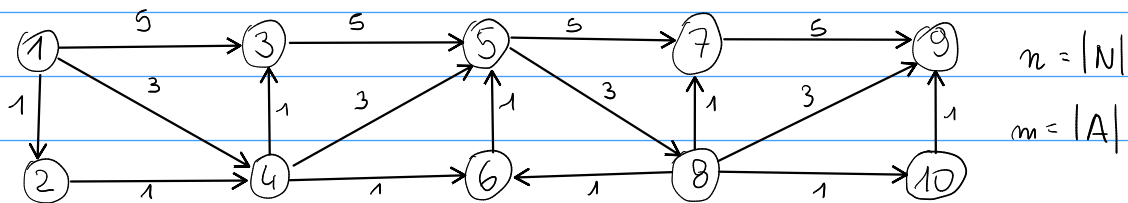
Gli archi che collegano S hanno capacità $u_{sj} = -b_j$ mentre quelli che collegano t $u_{it} = b_i$. Entrambi hanno costo $c_{sj} = c_{it} = 0$.

LIMITE di PIÙ NODI



PROBLEMI di CAMMINO di COSTO MINIMO (vedi pg. 8)

Overo un caso specifico del PROBLEMA di FLUSSO di COSTO MINIMO.



ESISTENZA della SOLUZIONE ① facendo la VISITA di un grafo (e.g. $b \rightarrow 1$ NO SOLUZIONE)
verifico la raggiungibilità di un nodo $O(m)$

② Se abbiamo un ciclo negativo allora il problema è
INFERIORMENTE ILLIMITATO e non ha soluzione

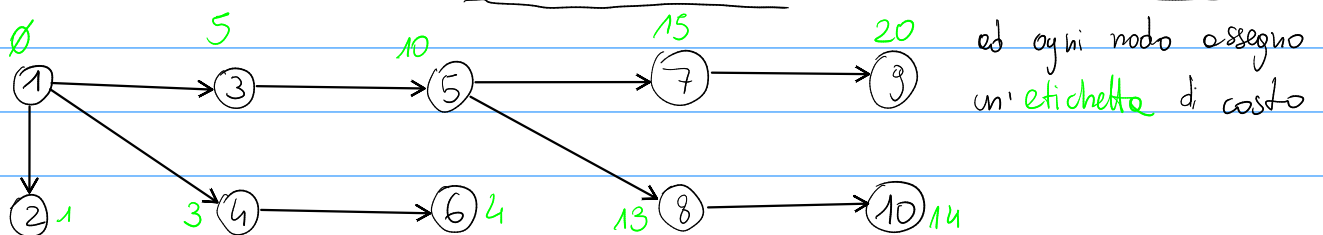


Note: dimostrazione a videolez. 7.1

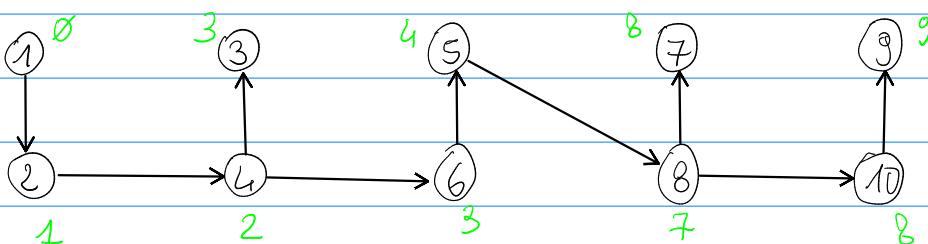
CAMMINO SEMPLICE ovvero non posso passare di nuovo da un nodo
MA! Stessa complessità del TSP

TEOREMA Dato un cammino minimo da i a j , tutti i sotto cammini sono anch'essi minimi
↓

In realtà mi serve trovare TUTTI i CAMMINI MINIMI. Posso strutturarli in un albero: dove



Per capire se è MINIMO devo provare a considerare gli archi lasciati fuori dall'albero
e vedere se il costo è minore. Dopo ogni cambiamento dell'albero devo aggiornare le etichette.



COME DIMOSTRIAMO CHE È EFFETTIVAMENTE L'ALBERO DEI CAMMINI MINIMI?

ALBERO
 $T = (N, A_T)$

ETICHETTE

$d[i]$

CONDIZIONI di BELLMAN

$$\forall (i,j) \in A \quad d[i] + c_{ij} \geq d[j]$$

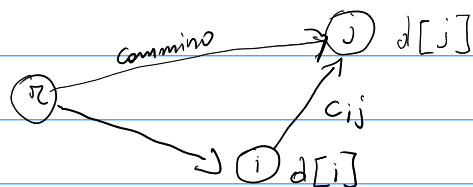
TEOREMA di BELLMAN Siano T un albero dei cammini e d il corrispondente vettore di etichette, allora T è un albero dei cammini minimi

SSÈ

Sono rispettate le CONDIZIONI di BELLMAN

DIMOSTRAZIONE

① $\Rightarrow$ ② Per ASSURDO



$$d[i] + c_{ij} < d[j] \quad \text{ma ovviamente non è possibile}$$

① $\Leftarrow$ ② Dato un albero che rispetta le condizioni di BELLMAN, è il migliore?

LEMMA Dato $d[i]$ che rispetta le CDB con $d[r] = 0$ allora

$$\forall P_{r,i} \quad c(P_{r,i}) \geq d[i]$$

DIMOSTRAZIONE $P_{r,i} \quad (i_1, i_2) \quad \text{CDB: } d[r] + c_{i_1 i_2} \geq d[i_2]$
 $i_1 = r \quad (i_2, i_3) \quad d[i_2] + c_{i_2 i_3} \geq d[i_3]$
 $i_k = i \quad (i_{k-1}, i_k) \quad \vdots \quad d[i_{k-1}] + c_{i_{k-1} i_k} \geq d[i_k]$

Sommando le diverse CDB rimane

$$d[r] + c(P_{r,i}) \geq d[i_k]$$

$$c(P_{r,i}) \geq d[i]$$

ALGORITMO SPT. L. QUEUE. BELLMAN

INPUT

Grafo G

Costi c_{ij}

Radice r

DEFINE

Vettore dei predecessori $p[i]$

Vettore delle etichette $d[i]$

FIFO Q (insieme, no duplicati)

INIT

Tutti i predecessori = r

Tutte le etichette = ∞

tranne $d[r] = 0$

$Q = \{r\}$

- Per ogni nodo $i \in Q$, finché Q ha elementi
 - Rimuovi i da Q
 - Per ogni arco che parte da i
 - Se $d[i] + c_{ij} < d[j]$
 - Aggiorna l'etichetta $d[j] = d[i] + c_{ij}$
 - Aggiorna il predecessore $p[j] = i$
 - Aggiungo il nodo alla queue $Q = Q \cup \{j\}$

NOTA: ORDINE LESSICOGRAFICO

TERMINAZIONE l'algoritmo NON può terminare quando non ci sono alberi di cammini minimi
ovvero quando ci sono cicli negativi

COMPLESSITÀ il caso peggiore è $O(n \cdot 2^n)$
 $\hookrightarrow$ Tutti i possibili cammini

PROPRIETÀ

Q è una FILA $\Rightarrow$ se $\nexists$ cicli orientati di costo negativo
allora nessun nodo può entrare in coda
più di $n-1$ volte

COMPLESSITÀ

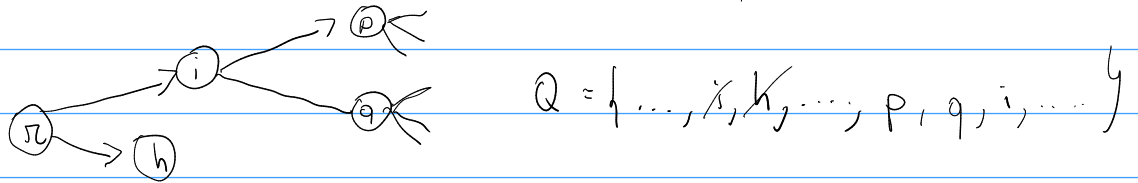
POLINOMIALE $O(n \cdot m)$

TERMINAZIONE

Se un nodo entra in Q n volte,
c'è un ciclo e devo terminare

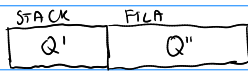
EFFICIENZA

non molto efficiente perché, essendo Q una FIFO, posso trovarmi in una situazione del tipo



dove io controllo p e q onde se so che dovrò rifarlo successivamente perché devo ricontrollare il predecessore i

STACK - DEQUE



per evitare ETICHETTE SPORCHE, se il nodo ha etichetta ∞ , è la prima volta che lo vedo e posso inserirlo in coda, altrimenti potrebbe aver già generato figli e quindi lo metto in testa

MA ! Perdo le proprietà della FILA per terminazione e ottimizzazione

2- QUEUE

Uso due code:

- "Sfigato" per i nodi non visitati, che non possono contenere etichette sporche
- "VIP" per i nodi già visitati

dove prima svuoto la coda VIP, Complessità $O(n^2 m)$

THRESHOLD

Definisco un valore T e in base a se l'etichetta è più grande o più piccola decido in quale lista piazzarlo (VIP o Sfigato)

ALGORITMO SPT.S

Utilizzo una LISTA di PRIORITÀ da cui estraggo i nodi in base all'etichetta minore.

CASO con ARCHI POSITIVI

LEMMA di DIJKSTRA Se $c_{ij} \geq 0 \quad \forall (ij) \in A$ **FONDAMENTALE!**

allora $d^k[i^k] \leq d^{k+1}[i^{k+1}] \quad \forall k$

Ovvero la sequenza delle etichette dei nodi che escono dalla coda ha sempre ordine crescente



TEOREMA di DIJKSTRA Ogni nodo esce dalla coda al più una volta

COMPLESSITÀ

$\boxed{n}$ ITERAZIONI

$O(n^2)$

Perché devo scorrere tutte le liste per trovare il minimo

$\boxed{m}$

QUANTE VOLTE VEDO UN ARCO

$O(m)$

$O(n^2 + m)$

GRAFO



DENSO

$m \approx n^2$

$O(n^2)$

SPARSO

$m \approx n$



Possono usare un

HEAP BINARIO BILANCIATO

• Estrazione del minimo $O(1)$

• Mantenimento ordine $O(\log n)$

• Inserimento nodo $O(\log n)$

$O(m \log n)$

NOTA !

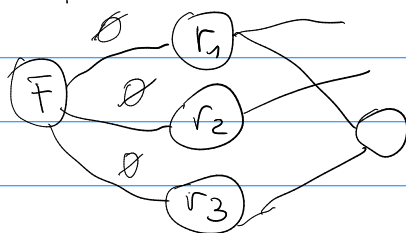
Quando i costi sono tutti positivi e vale il TEOREMA di DIJKSTRA non serve calcolare l'intero albero dei cammini minimi ma basta fermarsi quando esce il nodo destinazione

Cos'è successo se ci sono archi di costo negativo?

- Complessità ESPONENZIALE
- Devo usare un metodo diverso per il controllo dei CICLI NEGATIVI

CASO CON PIÙ RADICI

Creo un nodo finto collegato a tutte le radici con archi di costo 0 ed eseguo l'algoritmo partendo da lì.



CASO CON GRAFO ACICLICO

Come capisco che è Aciclico? BUONA NUMERAZIONE dei NODI

$$(i, j) \in A \quad i < j$$

Come si costruisce?

- ① $K=1$
- ② Prendo un nodo senza archi entranti
- ③ Gli do come nome K , $K++$
- ④ Elimino gli archi uscenti

Ora l'algoritmo consiste nello scorrere i nodi seguendo la BUONA NUMERAZIONE e aggiornando le etichette (NOTA! NO CYCLO)

NOTA! Se vogliamo come radice un nodo intermedio basta partire da lì

$$\left. \begin{array}{l} \text{COMPLESSITÀ} \\ \text{TROVARE BUONA NUMERAZIONE } O(m) \\ \text{APPLICAZIONE ALGORITMO } O(m) \end{array} \right\} O(m)$$

NOTA ! Se tutti gli archi sono negativi posso trovare il CAMMINO MASSIMO invertendo il segno dei costi e usando l'algoritmo del cammino minimo

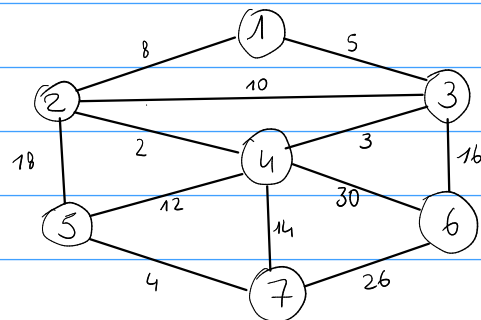
PROBLEMA DELL' ALBERO di COSTO MINIMO

Grafo $G = (V, E)$ VERTICI LATI NON ORIENTATO dove un vertice è $\{i, j\}$ con dei costi $c: E \rightarrow \mathbb{R}$

Vogliamo trovare un ALBERO di COPERTURA

$$T = (V, E_T) \quad E_T \subseteq E \quad |E_T| = n - 1$$

di COSTO $c(T) = \sum_{\{i,j\} \in E_T} c_{ij}$ MINIMO



ALGORITMO

Ho due sottoinsiemi:

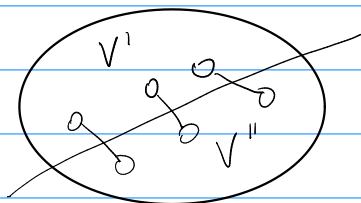
- $S \subseteq E$, lati che fanno parte della soluzione
- $R \subseteq E$, lati che NON fanno parte della soluzione

$$S \cap R = \emptyset$$

Inizio con $S = R = \emptyset$ e procedo con due possibili operazioni

INSERZIONE

Dato un taglio del tipo



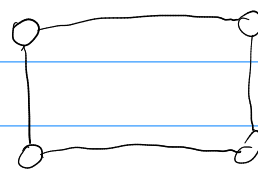
$$E(V', V'') \cap S = \emptyset$$

Devo scegliere il lato di costo minimo

$$\{i, j\} = \arg \min \{c_{ij} : \{i, j\} \in E(V', V'') \setminus R\}$$

CANCELLAZIONE

Dato un ciclo



$$C \subseteq E \quad C \cap R = \emptyset$$

Cancello il lato di costo massimo

$$\{i, j\} = \arg \max \{c_{ij} : \{i, j\} \in C \setminus S\}$$

TEOREMA Partendo da una scelta di S e R che non uccide tutte le soluzioni ottime, entrambe le operazioni non cambiano queste proprietà

↓

ALGORITMO

GREEDY

$$S = \emptyset ; R = \emptyset ;$$

do INSERZIONE. CANCELLAZIONE

$$\text{while } (S \cup R \neq E) \text{ AND } (|S| < n - 1)$$

ALGORITMO di KRUSKAL

$S = \emptyset; R = \emptyset; X = \text{LATI ORDINATI}$

do

estrai $\{u, v\}$ da X
 se forma ^{log m} un ciclo $R = R \cup \{u, v\}$
 altrimenti $S = S \cup \{u, v\}$

while $|S| < n - 1$

COMPLESSITÀ $O(m \log m)$

ALGORITMO di PRIM

INPUT

Grafo G

Costi c_{ij}

Radice r

DEFINE

Vettore dei predecessori $p[i]$

Vettore delle etichette $d[i]$

Lista Q (insieme, no duplicati)

INIT

Tutti i predecessori = r

Tutte le etichette = ∞

tranne $d[r] = -\infty$ |
 $Q = \{r\}$

- Per ogni nodo $i \in Q$, prendo quello di etichetta MINIMA
 - Rimuovo i da Q ; $d[i] = -\infty$; (corrisponde all'inserimento)
 - Per ogni arco che incrocia i
 - se $c_{ij} < d[j]$
 - Aggiorno l'etichetta $d[j] = c_{ij}$
 - Aggiorno il predecessore $p[j] = i$
 - Aggiungo il nodo alla queue $Q = Q \cup \{j\}$

NOTA! Simile a SPT.S ma DIVERSO!

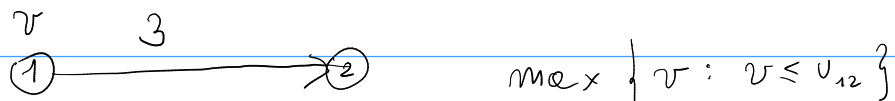
COMPLESSITÀ

DENSO $O(n^2)$

SPARSO — HEAP BINARIO BILANCIATO $O(m \log n)$

PROBLEMA del FLUSSO MASSIMO

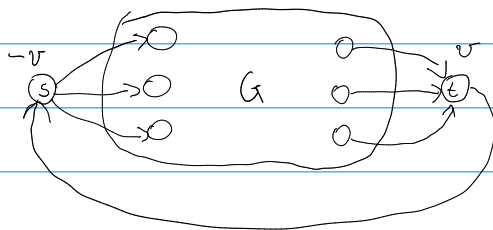
Il problema del flusso massimo è DECISIONALE, quindi per averne uno di ottimizzazione ci chiediamo invece il valore massimo v che può passare per un cammino



Vogliamo il massimo valore v del Flusso X che soddisfa

$$\sum_{(j,i) \in BS(i)} X_{ji} - \sum_{(i,j) \in FS(i)} X_{ij} = \begin{cases} -v & i=s \\ v & i=t \\ 0 & \text{altrimenti} \end{cases} \quad i \in N$$

Posso anche vederlo come problema di FLUSSO di COSTO MINIMO



aggiungendo un arco di ritorno da t ad s con costo -1 e con tutti gli altri costi pari a 0 .

A questo punto minimizzare $-v$ corrisponde a massimizzare v l'ungo l'arco di ritorno.

CAMMINO AUMENTANTE

Cerchiamo un CAMMINO AUMENTANTE, ovvero un cammino P che va da s a t e non è orientato. $P = P^+ \cup P^-$, $P^+ \cap P^- = \emptyset$

e con CAPACITÀ $\theta(P, x) = \min \begin{cases} \min \{ v_{ij} - x_{ij} : (ij) \in P^+ \} \\ \min \{ x_{ij} : (ij) \in P^- \} \end{cases}$

DEFINIZIONE P è AUMENTANTE $\Leftrightarrow \theta(P, x) > 0$

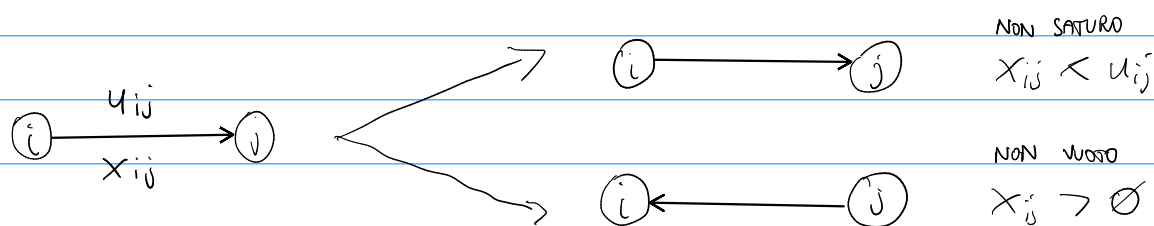
Otengo poi un nuovo Flusso tramite COMPOSIZIONE tra FLUSSI e CAMMINI

$$X' = X \oplus (\theta)P \quad \theta \in \mathbb{R}$$

$$X'_{ij} = \begin{cases} X_{ij} + \theta & (ij) \in P^+ \\ X_{ij} - \theta & (ij) \in P^- \\ X_{ij} & (ij) \notin P \end{cases}$$

TEOREMA Dato un Flusso ammissibile X di valore v , un cammino P di capacità $\sigma(P, X)$ ed un numero $0 \leq \theta \leq \sigma(P, X)$ allora $X' = X \oplus \theta P$ è un Flusso ammissibile di valore $v + \theta$ ovvero $0 \leq X'_{ij} \leq u_{ij} \quad (ij) \in A$

GRAFO RESIDUO RISPETTO AD UN FLUSSO AMMISSIBILE X



TEOREMA $\forall$ CAMMINO AUMENTANTE P rispetto ad X , $\exists!$ CAMMINO ORIENTATO da s a t sul GRAFO RESIDUO

Quindi per trovare CAMMINI AUMENTANTI è sufficiente fare una visita del GRAFO RESIDUO

OTTIMALITÀ di un FLUSSO

Consideriamo un taglio (N_s, N_t) $N_s \cup N_t = N$ $s \in N_s$
 $N_s \cap N_t = \emptyset$ $t \in N_t$

e gli archi che lo attraversano $A^+(N_s, N_t) = \{(ij) : i \in N_s, j \in N_t\}$
 $A^-(N_s, N_t) = \{(ij) : i \in N_t, j \in N_s\}$

Definiamo il FLUSSO che lo attraversa $X(N_s, N_t) = \sum_{(ij) \in A^+(N_s, N_t)} x_{ij} - \sum_{(ij) \in A^-(N_s, N_t)} x_{ij}$

e la massima quantità di flusso che lo può attraversare $u(N_s, N_t) = \sum_{(ij) \in A^+(N_s, N_t)} u_{ij}$

TEOREMA

$\forall$ X FLUSSO AMMISSIBILE di valore v

$\forall$ TAGLIO (N_s, N_t)

$$v = X(N_s, N_t) \leq u(N_s, N_t)$$

DIMOSTRAZIONE

Partiamo dai vincoli di conservazione del flusso

$$\sum_{(j,i) \in BS(i)} x_{ji} - \sum_{(i,j) \in FS(i)} x_{ij} = \begin{cases} -v & i=s \\ v & i=t \\ 0 & \text{altrimenti} \end{cases} \quad i \in N$$

Ora prendo le $i \in N_t$ e le sommo

$$\sum_{i \in N_t} \left(\sum_{(j,i) \in BS(i)} x_{ji} - \sum_{(i,j) \in FS(i)} x_{ij} \right) = v$$

e infine

$$\sum_{i \in N_t} \left(\sum_{(j,i) \in BS(i)} x_{ji} - \sum_{(i,j) \in FS(i)} x_{ij} \right) = \sum_{(ij) \in A^+(N_s, N_t)} x_{ij} - \sum_{(ij) \in A^-(N_s, N_t)} x_{ij} = X(N_s, N_t)$$

dato che posso avere 4 casi

- $i, j \in N_s \rightarrow$ Non viene sommato
- $j \in N_s, i \in N_t \quad j \rightsquigarrow i \rightarrow$ Sommo
- $j \in N_s, i \in N_t \quad i \rightsquigarrow j \rightarrow$ Sottraggo
- $i, j \in N_t \rightarrow$ Sommo e sottraggo = $\emptyset$

Inoltre per definizione

$$\sum_{(ij) \in A^+(N_s, N_t)} x_{ij} \leq \sum_{(ij) \in A^+(N_s, N_t)} u_{ij} = u(N_s, N_t)$$

e sottraendo mantengo valida la disuguaglianza

$$X(N_s, N_t) = \sum_{(ij) \in A^+(N_s, N_t)} x_{ij} - \sum_{(ij) \in A^-(N_s, N_t)} x_{ij} \leq \sum_{(ij) \in A^+(N_s, N_t)} u_{ij} = u(N_s, N_t)$$

Quindi per capire se un Flusso è OTTIMALE devo trovare un taglio per cui vale l'UGUAGLIANZA

$$v = \sum (N_s, N_t) = u(N_s, N_t)$$

e questo succede quando abbiamo un TAGLIO SATURO

- Gli archi CONCORDI sono SATURI

$$\sum_{(ij) \in A^+(N_s, N_t)} x_{ij} = \sum_{(ij) \in A^+(N_s, N_t)} u_{ij} = u(N_s, N_t)$$

- Gli archi DISCORDI sono VUOTI

$$\sum_{(ij) \in A^-(N_s, N_t)} x_{ij} = 0$$

PROBLEMI di INTERDIZIONE OTTIMA

Risolvendo il problema di OTTIMITÀ di un FLUSSO MASSIMO ho anche trovato la soluzione per trovare il TAGLIO di CAPACITÀ MINIMA

$$\min \left\{ u(N_s, N_t) : \begin{array}{l} N_s \cup N_t = N \quad s \in N_s \\ N_s \cap N_t = \emptyset \quad t \in N_t \end{array} \right\}$$

dato che se troviamo il flusso massimo, la capacità del taglio u è $\mathbb{Z}$.

TERMINAZIONE

L'algoritmo termina se le capacità sono INTERE

$$u \in \mathbb{N} \rightarrow x \in \mathbb{N} \quad \text{perché} \quad x' = v + \sigma \quad v \in \mathbb{N}$$

$$\sigma = u_{ij} - x_{ij} \quad u, x \in \mathbb{N}$$

e partiamo da $v = 0$

COMPLESSITÀ

Il caso peggiore è quando in E entrano $n-1$ archi di capacità

$$U = \max \{u_{ij} : (i,j) \in E\}$$

In quel caso $v = 0 \dots U$ dove ad ogni iterazione aumento v di 1.
Per ogni iterazione devo fare la visita del GRAFO RESIDUO ($O(n)$)

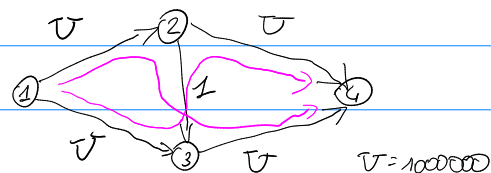
QUINDI

$$O(n m U)$$

È **PSEUDO POLINOMIALE** in quanto diventa **ESPONENZIALE** se i numeri dell'istanza sono grandi. L'istanza si può però rappresentare **POLINOMIALMENTE** $O(m \log U)$

↳ **NON TANTI**

MA GRANDI e PICCOLI $\rightarrow$



Una possibile **OTTIMIZZAZIONE** è mettere U in AND, scartando i bit con capacità basse, eseguire l'algoritmo e poi ripetere aggiungendo un bit

$$U(5\text{bit}) \text{ AND } 10000$$

Prendo i bit con $u_{ij} \geq 32$

PROSSIMA ITERAZ.

$$\rightarrow U(5\text{bit}) \text{ AND } 11000$$

Prendo i bit con $u_{ij} \geq 16$

$$O(m^2 \log U) \quad \text{POLINOMIALE}$$

ALGORITMO di EDMONDS & KARP

Faccio la visita del GRAFO RESIDUO usando una coda FIFO come in SP.T.L. QUEUE. Questo ci garantisce che partiremo dai CAMMINI AUMENTANTI più corti e finiremo con quelli più lunghi

COMPLESSITÀ

Ogni arco si setta o si svuota al più n volte
Ogni visita costa m

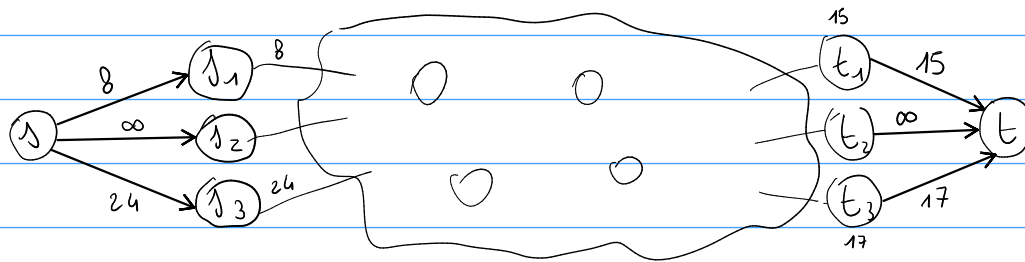
$$O(n m^2)$$

! NON DIPENDE DA U

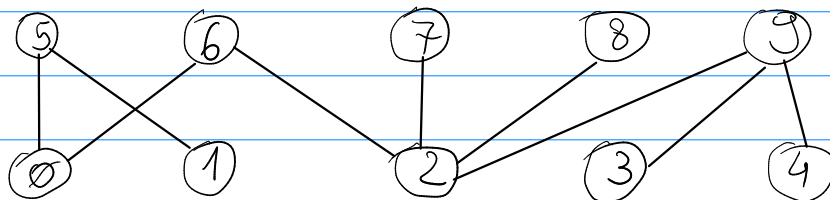
Se il grafo è denso $O(n^5)$

SORGENTI e POZZI MOLTEPLICI

È sufficiente aggiungere una sorgente generale e un pozzo generale.



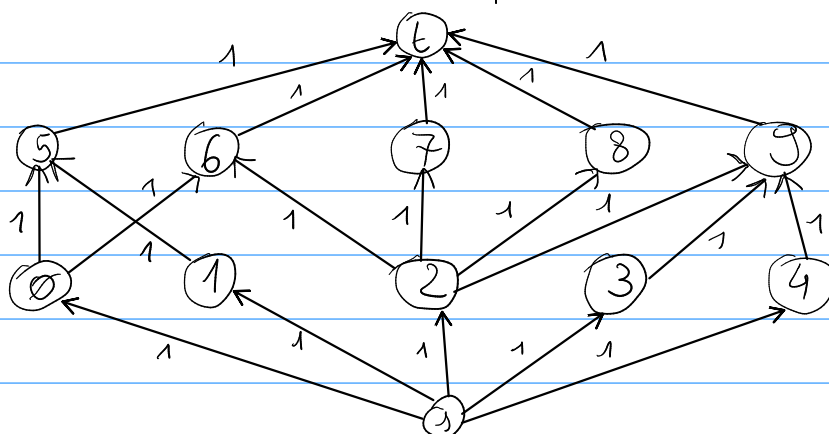
PROBLEMA dell'ACCOPIAMENTO di MASSIMA CARDINALITÀ



I nodi sopra vogliono parlare con quelli sotto ma non tutti sono disposti a parlare con tutti

Un nodo sopra può essere accoppiato con un solo nodo sotto e vogliamo ottenere il numero MASSIMO di COPPIE possibili.

Lo trasformiamo in un problema di FLUSSO MASSIMO



Nei lati interni se passa del Flusso $\rightarrow \exists$ accoppiamento

$$|coppie| = \text{flusso} \times$$

COMPLESSITÀ $O(n m U)$ $U=1$

$$O(n m)$$

PROBLEMA di FLUSSO di COSTO MINIMO

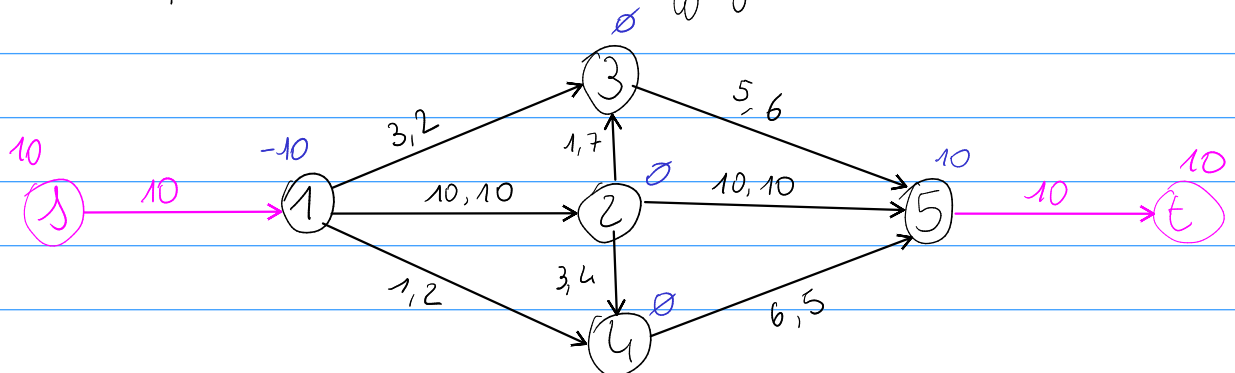
Problema dei
CAMMINI MINIMI

c_{ij}

+ Problema del
FLUSSO MASSIMO

u_{ij}

Come implementiamo i DEFICIT? Aggiungo s e t



Esiste sempre la soluzione? NO! Si vede applicando il Flusso massimo

Una volta applicato il flusso massimo, possiamo aumentarlo modificando le capacità degli archi che attraversano il TAGLIO SATURO

N.B. NON BISOGNA FERMARSI AL PRIMO TAGLIO SATURO, DOPO AVER "DESATURATO" QUELLO POTREBBERO ESSERCENE ALTRI !!!

PSEUDOFUSSO Vettore $x = [x_{ij}]_{(ij) \in A}$ che rispetta i vincoli di capacità
 $0 \leq x_{ij} \leq u_{ij} \quad \forall (ij) \in A$

SBILANCIAMENTO
di un NODO

$$e_x(i) = \sum_{(ji) \in BS(i)} x_{ji} - \sum_{(ij) \in FS(i)} x_{ij} - b_{ij}$$

$= 0$ BILANCIATO
 > 0 SORGENTE O_x
 < 0 POZZO D_x

Se $O_x = 0 = D_x$ allora abbiamo un FLUSSO

Consideriamo i cammini P_{st} , $P = P^+ \cup P^-$ con capacità $\theta(P, x)$ e la composizione tra Flussi e cammini $x' = x \oplus \theta P$ (pagine 22-23)

Allora lo SBILANCIAMENTO dei Nodi vale

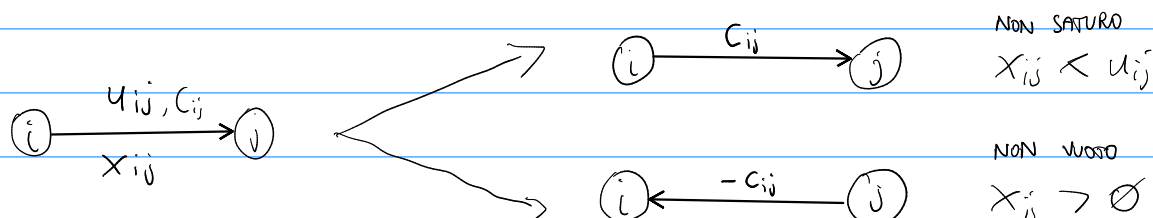
$$e_{x'}(i) = \begin{cases} e_x(i) - \theta & i = s \\ e_x(i) + \theta & i = t \\ e_x(i) & i \notin \{s, t\} \end{cases}$$

Prendendo $s=t$ ottengo un **CICLO AUMENTANTE** $P = C$ e gli sbilanciamenti dei nodi rimangono 0, facendo in modo che lo PSEUDOFUSSO rimanga AMMISSIBILE (FLUSSO).

$$x' = x \oplus \theta C \quad \theta \leq \theta \leq \theta(C, x)$$

COSTI $C(P) = \sum_{(i,j) \in P^+} c_{ij} - \sum_{(i,j) \in P^-} c_{ij} \quad C(x') = C(x) + \theta \cdot C(C)$

Come trovo un CICLO AUMENTANTE? **GRAFO RESIDUO**



$\forall$ ciclo AUMENTANTE P rispetto ad x , $\exists!$ ciclo ORIENTATO da s a t sul GRAFO RESIDUO con lo stesso costo

ALGORITMO

① Trovo un FLUSSO MASSIMO

② Cerco cicli AUMENTANTI di COSTO NEGATIVO e ci mando FLUSSO

Come li trovo? **SPT.L. QUEUE** che se termina con Q vuoto vuol dire che non ci sono cicli e ho terminato, altrimenti ho trovato un ciclo con una certa capacità e posso mandarci Flusso

TERMINAZIONE

TEOREMA Dati due PSEUDOFUSSI x' e x'' , $\exists K = m + m$ cicli o CAMMINI
di DECOMPOSIZIONE AUMENTANTI $P_1, P_2, \dots, P_K$ (al massimo m cicli) rispetto ad x'
degli PSEUDOFUSSI e K numeri $0 < \theta_i \leq \theta(P_i, x')$ tali che

$$x'' = x' \oplus \theta_1 P_1 \oplus \theta_2 P_2 \oplus \dots \oplus \theta_K P_K$$

e i P_i che sono CAMMINI hanno origine s_i e destinazione t_i tale
che $e_{x'}(s_i) \neq e_{x''}(s_i)$ e $e_{x'}(t_i) \neq e_{x''}(t_i)$

COROLLARIO Se x' e x'' sono FLUSSI AMMISSIBILI allora tutti i P_i sono CICLI

TEOREMA X è un FLUSSO OTTIMO $\Leftrightarrow \nexists$ cicli AUMENTANTI di costo negativo

DIMOSTRAZIONE • Supponiamo di avere X NON OTTIMO e $\nexists$ cicli AUMENTANTI di COSTO NEGATIVO

• Allora esiste una soluzione migliore x^* per cui $C(x^*) < C(x)$

• Uso il teorema di DECOMPOSIZIONE degli PSEUDOFUSSI

$$x^* = x \oplus \theta_1 C_1 \oplus \theta_2 C_2 \oplus \dots \oplus \theta_K C_K$$

$$C(x^*) = C(x) + \theta_1 C(C_1) + \dots + \theta_K C(C_K)$$

• Per definizione di ciclo aumentante $\theta_K > 0$ e $C(C_K) > 0$

• Quindi $C(x^*) > C(x)$ che va contro l'ipotesi ■

NOTA! Per cercare i cicli sul grafo resi due devo usare tutti i nodi come quello di partenza, mettendo a 0 tutte le etichette.

CONDIZIONI

$$\underbrace{u \in \mathbb{N}_+^m \quad b \in \mathbb{Z}^n \quad c \in \mathbb{Z}^m}_{x \in \mathbb{N}^m}$$

COMPLESSITÀ

$$U = \max \{ u_{ij} : (ij) \in A \} \quad \text{MASSIMA CAPACITÀ}$$

$$\bar{c} = \max \{ \max \{ 0, c_{ij} \} : (ij) \in A \} \quad \text{COSTO MASSIMO degli archi POSITIVI}$$

$$\underline{c} = \min \{ \min \{ 0, c_{ij} \} : (ij) \in A \} \quad \text{COSTO MINIMO degli archi NEGATIVI}$$

Il valore ottimo è quindi compreso tra

$$\underline{c} \cdot m \cdot U \leq Z(\text{MCF}) \leq \bar{c} \cdot m \cdot U$$

Ad ogni iterazione troviamo un ciclo negativo che è per forza intero $C(C_1) \leq -1$ e ci mandiamo una quantità di flusso intero $\theta = \theta(C_1, x) \geq 1$

Quindi $C(x') = C(x) + \theta C(C_1) \leq C(x) - 1$ ovvero ad ogni iterazione il costo diminuisce di 1 nel caso PEGGIORE.

$$O \left(\underbrace{(\bar{c} - \underline{c}) \cdot m \cdot U}_{\text{ITERAZIONE}} \cdot \underbrace{nm}_{\text{SPT.L. QUEUE}} \right)$$

OTTIMIZZAZIONE

- ① Si possono trovare cicli con capacità grande e costo basso
- ② Si può applicare il BIT FILTER (pagine 26)
- ③ Invece di usare SPT.L. QUEUE usiamo un algoritmo più efficiente

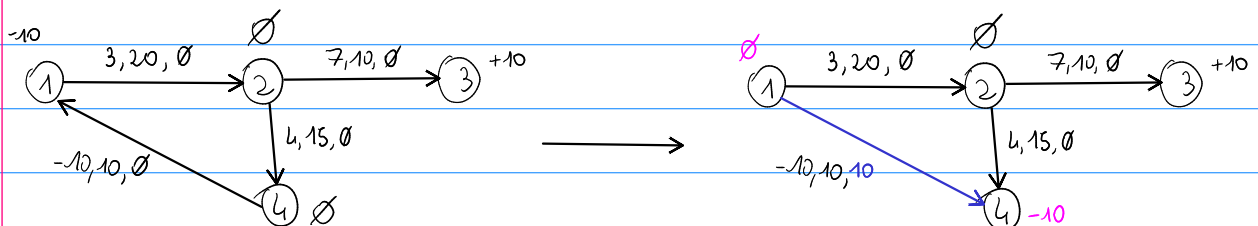
- ④ Trovo una migliore soluzione di partenza: trovo il flusso MASSIMO usando CAMMINI MINIMI per cercare sul GRAFO RESIDUO

ALGORITMO DEI CAMMINI MINIMI SUCCESSIVI

PROBLEMA: potrebbe esserci un ciclo orientato di costo negativo

SOLUZIONE: saturare tutti gli archi di costo negativo

Questo significa che ottergo un PSEUDOFUSSO diverso dove i nodi con DEFICIT non BILANCIATI sono cambiati



Abbiamo ottenuto uno

PSEUDOFUSSO MINIMALE X con vettore degli sbilanciamenti e_X tale che $\forall x', e_{x'} = e_X$ vale $c(x) \leq c(x')$

↓

Un FLUSSO OTTIMO è un particolare PSEUDOFUSSO MINIMALE con il vettore degli sbilanciamenti $e = 0$

TEOREMA X è uno PSEUDOFUSSO MINIMALE $\Leftrightarrow \nexists$ CICLI AUMENTANTI di costo NEGATIVO

TEOREMA Sia X uno PSEUDOFUSSO MINIMALE, P un CAMMINO AUMENTANTE da s a t di costo minimo

Allora $\forall 0 \leq \theta \leq \theta(P, X)$, $x' = X \oplus \theta P$ è MINIMALE

DIMOSTRAZIONE X'' con $e_{X''} = e_{X'}$

Applico il TEOREMA di DECOMPOSIZIONE degli PSEUDOFUSSI

$$X'' = X \oplus \underbrace{\theta_1 P_1 \oplus \dots \oplus \theta_h P_h}_{\text{CAMMINI AUMENTANTI}} \oplus \underbrace{\theta_{h+1} C_1 \oplus \dots \oplus \theta_k C_k}_{\text{CICLI AUMENTANTI}}$$

$P_1, \dots, P_m$ vanno da s a t

- $C(P_i) \geq C(P)$ Perché P era il cammino accumulato di costo minimo e quindi per definizione P_i non può costare meno
- $\theta = \theta_1 + \dots + \theta_m$ perché gli sbilanciamenti devono essere uguali

Quindi il costo è

$$\geq \theta C(P)$$

$$C(x'') = C(x) + \theta_1 C(P_1) + \dots + \theta_m C(P_m) + \theta_{m+1} C(G_1) + \dots + \theta_k C(G_k) \geq 0$$

Perché sto lavorando su PSEUDOFUSSI MINIMALI che quindi non hanno cicli NEGATIVI

$$\text{Quindi } C(x'') \geq C(x) + \theta C(P) \\ C(x'') \geq C(x')$$

ALGORITMO

- INIZIALIZZAZIONE Ogni arco positivo ha flusso $x_{ij} \neq 0$ | PSEUDOFUSSE
negativo $x_{ij} = u_{ij}$ | MINIMALE
- Finché il deficit totale è $\neq 0$
 - Trovare i CAMMINI MINIMI tra s (DEFICIT NEGATIVO) e t (DEFICIT POSITIVO) sul grafo residuo
 - Prendo il minimo tra i cammini minimi guardando l'etichetta dei nodi
 - Se l'etichetta è ∞ , non c'è il cammino minimo e termino
 - Aumento il flusso sul cammino $\theta = \min \{ \theta(P, x), e_x(s), -e_x(t) \} > 0$

CORRETTEZZA Abbiamo dimostrato nel teorema precedente che non possono esserci cicli di costo negativo

TERMINAZIONE Devo avere capacità e deficit INTERI $u \in \mathbb{N}_{++}^m$ $b \in \mathbb{Z}^n$
così da partire da un flusso intero $x \in \mathbb{N}^m$
e mi aggiungo sempre un intero $\theta = \min \{ \theta(P, x), e_x(s), -e_x(t) \}$
e quindi tutti i flussi sono interi e ad ogni iterazione aumento di $\theta \geq 1$
 $e(x) \in \mathbb{N}$, $e(x) \leq [mV + mB]$ diminuisce almeno di 1

COMPLESSITÀ

$$O(mV \log m)$$

→ SPT. L. QUEUE perché gli archi possono avere costi negativi

OPPURE uso il ETTORE di POTENZIALI $[\pi_i]_{i \in N}$ con il COSTO RIDOTTO $C_{ij}^\pi = c_{ij} - \pi_i + \pi_j$
ricordando che $d_j \leq c_{ij} + d_i$ e $c_{ij} + \pi_i - \pi_j \geq 0$, se metto $\pi = -d$ ottengo $C_{ij}^\pi \geq 0$
posso applicare SPT.S che è $O(m \log n)$

PROBLEMI DI PROGRAMMAZIONE LINEARE

Voglio trovare il valore di VARIABILI $X = [x_i]_{i=1 \dots n} \in \mathbb{R}^n$ dati:

- FUNZIONE OBIETTIVO $\begin{cases} \min \\ \max \end{cases} \left\{ \sum_{j=1}^n c_j x_j \right\} \quad c = [c_i]_{i=1 \dots n} \in \mathbb{R}^n$

- VINCOLI $\sum_{j=1}^n a_{ij} x_j \begin{cases} \geq \\ \leq \\ = \end{cases} b_i \quad i = 1 \dots m$

che possono essere scritti come VETTORI $A_i x \begin{cases} \geq \\ \leq \\ = \end{cases} b_i \quad i = 1 \dots m$

FORMA NORMALE $\max \{ c x : A x \leq b \}$
 $c \in \mathbb{R}^n \quad A \in \mathbb{R}^{m \times n} \quad b \in \mathbb{R}^m$

FORMA ALTERNATIVA $\min \{ c x : A x = b, x \geq 0 \}$

TRASFORMAZIONI EQUIVALENTI

• se $a \geq b$ allora $-a \leq -b$

• $a = b$ corrisponde ad $a \leq b \wedge a \geq b$

$a \leq b \wedge -a \leq -b$

• $\max \{ f(x) \} = -\min \{ -f(x) \}$

• Per passare da DISEQUAZIONI a EQUAZIONI uso le VARIABILI di SCARTO

$4x_A + x_B + 3x_C \geq 32 \rightarrow 4x_A + x_B + 3x_C - s_1 = 32 \quad s_1, s_2 \geq 0$

$4x_A + x_B + 3x_C \leq 55 \rightarrow 4x_A + x_B + 3x_C + s_2 = 55$

• Se $x \geq 0$ posso scriverla come $x = x^+ - x^- \quad x^+, x^- \geq 0$
NON VINCOLATA IN SEGNO

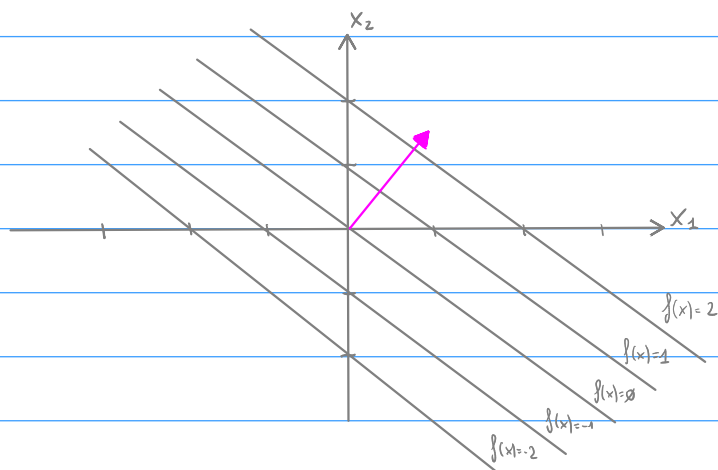
INTERPRETAZIONE GEOMETRICA

Abbiamo una FUNZIONE LINEARE del tipo $f(x) = a \cdot x = \sum_{i=1}^m a_i x_i$ o eventualmente una FUNZIONE LINEARE AFFINE $f(x) = a \cdot x + b = \sum_{i=1}^m a_i x_i + b$

Per rappresentarle usiamo le CURVE di LIVELLO $L_k(f) = \{x \in \mathbb{R}^m : a \cdot x = k\}$

Esempio $f(x) = x_1 + x_2 = \begin{bmatrix} 1, 1 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix}$

IPERPIANO



NOTA In termini di PRODOTTO SCALARE

$$f(x) = \langle a, x \rangle = \|a\| \cdot \|x\| \cdot \cos(\theta)$$

- $\xrightarrow{x} a$ $\theta = 0, \cos \theta = 1$
- $\nearrow x$ $0 < \theta < \frac{\pi}{2}, 0 < \cos \theta < 1$
- $\downarrow x$ $\theta = \frac{\pi}{2}, \cos \theta = 0$
- $\nwarrow x$ $\pi < \theta < \frac{3\pi}{2}, -1 < \cos \theta < 0$
- $\xleftarrow{x} a$ $\theta = \pi, \cos \theta = -1$

quando abbiamo una CURVA di SOTTO LIVELLO $L_k(f) = \{x \in \mathbb{R}^m : a \cdot x \leq k\}$ abbiamo un SEMISPACIO

Tramite i VINCOLI $I = \{1, \dots, m\}$ definiamo un POLIEDRO $\mathcal{P} = \{x \in \mathbb{R}^m : A \cdot x \leq b\}$.

Dato un sottoinsieme di vincoli $S \subseteq I$, definisco una FACCIA come $\mathcal{P}_S = \{x \in \mathbb{R}^m : A_i x = b_i \quad i \in S, A_i x \leq b_i \quad i \in I/S\}$

Le Facce Formate da un singolo vincolo sono chiamate MASSIMALI e non sono contenute da facce più grosse di loro

Fatta eccezione per i VINCOLI RIDONDANTI.

Utilizzando m vincoli (e.g. $m=3$ per 3D) ottengo FACCE MINIMALI o VERTICI

INVOLUPPO CONVESSO dati due punti $x, z \in \mathbb{R}^m$ è $\text{CONV}(\{z, x\}) = \{w = \alpha x + (1-\alpha)z : \alpha \in [0, 1]\}$

ovvero il segmento che li unisce

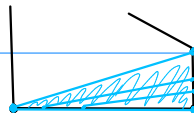


Posso definire il mio poliedro come INVOLUPPO CONVESSO di tutti i VERTICI definiti dai vincoli

$$X \subset \mathbb{R}^m = \{x_1, x_2, \dots, x_k\}$$

$$\text{CONV}(X) = \left\{ x = \sum_{i=1}^k \alpha_i x_i : \sum_{i=1}^k \alpha_i = 1, \alpha_i \geq 0 \quad i=1 \dots k \right\}$$

POLITOPPO



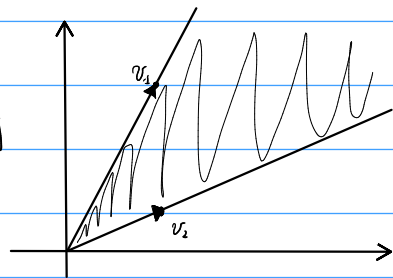
NOTA Massimizzare un poliedro definito come INVOLUPPO CONVESSO è molto facile

$$\max \left\{ c \left(\sum_{i=1}^k \alpha_i x_i \right) = \sum_{i=1}^k \alpha_i c(x_i) : \sum_{i=1}^k \alpha_i = 1, \alpha_i \geq 0 \quad i=1 \dots k \right\}$$

basta prendere la x_i con maggiore $c(x_i)$ e porre $\alpha_i = 1$

INVILUPPO CONICO

Dato un insieme di punti $V = \{v_1, v_2, \dots, v_h\} \subset \mathbb{R}^n$
 definisco CONO(V) = $\{v = \sum_{i=1}^h \beta_i v_i : \beta_i \geq 0 \quad i=1, \dots, h\}$
CONO FINITAMENTE GENERATO
 NOTA: a differenza dell'INVILUPPO CONVESSO, $\sum_{i=1}^h \beta_i = k$



TEOREMA di DECOMPOSIZIONE DEI POLIEDRI

Posso rappresentare un POLIEDRO $P = \{x \in \mathbb{R}^n : Ax \leq b\}$ come

$$\text{CONV}(X) + \text{CONO}(V)$$

$$X = \{x_1, \dots, x_k\} \subset \mathbb{R}^n$$

$$V = \{v_1, \dots, v_h\} \subset \mathbb{R}^n$$

→ SOMMA di INSIEMI $A+B = \{v = a+b : a \in A, b \in B\}$

Posso scriverlo anche come

$$x = \sum_{i=1}^k \alpha_i x_i + \sum_{j=1}^h \beta_j v_j$$

$$\sum_{i=1}^k \alpha_i = 1$$

$$i=1 \dots k$$

$$\alpha_i, \beta_j \geq 0$$

$$j=1 \dots h$$

Anche in questo caso, massimizzare si traduce in

$$\max \{c(x)\} = \max \left\{ \sum_{i=1}^k \alpha_i c(x_i) + \sum_{j=1}^h \beta_j c(v_j) \right\}$$

NOTA ! Se abbiamo un qualche $c(v_j) > 0$ vuol dire che
 il nostro problema è SUPERIORMENTE ILLIMITATO

PROBLEMA !

Il numero di vertici cresce esponenzialmente con le dimensioni n
 e.g. per un IPERCUBO facce = $2 \cdot n$ vertici = 2^n

CASO SENZA VERTICI

VERTICE Un punto del poliedro che non può essere scritto come COMBINAZIONE CONVESSA di altri punti diversi del poliedro

DIREZIONE del POLIEDRO $v \in \mathbb{R}^n$ tale che $\forall x \in P, \forall \alpha \geq 0 \quad x + \alpha v \in P$

CONO di RECESSIONE C è un cono $\Leftrightarrow \forall x \in C, \forall \alpha \geq 0 \quad \alpha x \in C$

$$\left. \begin{array}{l} \text{CONO(V)} = \text{rec(P)} = \{v \in \mathbb{R}^n : v \text{ è una direzione}\} = \\ = \{v \in \mathbb{R}^n : Av \leq 0\} \end{array} \right\}$$

NOTA ! Un CONO FINITAMENTE GENERATO è un POLIEDRO esso stesso e può quindi essere scritto nella forma

$$\text{CONO(V)} = \{x : Ax \leq 0\} \longrightarrow \text{CONO POLIEDRICO}$$

DIREZIONE di LINEALITÀ

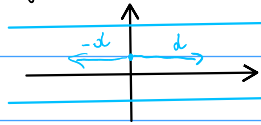
d è una DIREZIONE di LINEALITÀ di $P \Leftrightarrow d \in \text{rec}(P) \wedge -d \in \text{rec}(P)$

$$Ad \leq 0 \wedge A(-d) \leq 0$$

$$Ad \leq 0 \wedge Ad \geq 0$$

$$\boxed{Ad = 0}$$

Se ci sono DIREZIONI di LINEARITÀ vuol dire che non abbiamo vertici perché possiamo andare avanti quanto vogliamo sia lungo la direzione d che lungo la direzione $-d$



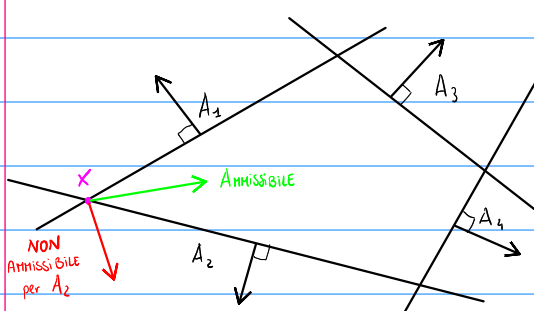
Quando questo succede vuol dire che la matrice A NON HA RANGO di COLONNA PIENO e vuol dire che almeno una delle colonne è LINEARMENTE DIPENDENTE dalle altre. $A = [A', A''] \rightarrow A'v = A''$

Questo implica anche che $C = [C', C'']$ e $X = [X', X'']$ e quindi posso eliminare quella variabile e risolvere il problema di massimizzazione più semplice $P' = \max \{C'x' : A'x' \leq b'\}$. Questo può avere 3 risultati:

- $\exists$ SOLUZIONE AMMISSIBILE $\bar{x}'$ per $P' \Rightarrow [\bar{x}', 0]$ è AMMISSIBILE per P e ha lo stesso costo e quindi P' è SUPERIORMENTE ILLIMITATO $\Rightarrow P$ è SUPERIORMENTE ILLIMITATO
- (P') è VUOTO $\Rightarrow (P)$ è VUOTO
- $\exists$ SOLUZIONE OTTIMA x^{*1} per P' , allora per definizione $\exists$ una direzione verso la quale ci si può allontanare rimanendo sempre nel poliedro P di P , ovvero $A'v = A'' [A', A''] \begin{bmatrix} v \\ -1 \end{bmatrix} = 0$ e quindi $\bar{x} = [x^{*1}, 0]$ AMMISSIBILE. Abbiamo quindi due casi:
 - $cd \neq 0 \Rightarrow P$ è SUPERIORMENTE ILLIMITATO
 - $cd = 0 \Rightarrow \bar{x}$ è OTTIMA per P

DIREZIONE AMMISSIBILE d è DIREZIONE AMMISSIBILE per $x \in P$ se $\exists \bar{\alpha} > 0 : x + \alpha d \in P \quad \forall \alpha \in [0, \bar{\alpha}]$

Come sono fatte? $Ax \leq b \rightarrow A_i x \leq b_i \quad i=1 \dots m$



$$A(x + \alpha d) \leq b \rightarrow A_i(x + \alpha d) \leq b_i$$

$$A_i x + \alpha(A_i d) \leq b_i$$

$$\alpha \geq 0$$

$$\alpha(A_i d) \leq b_i - A_i x$$

$$\alpha \leq \frac{b_i - A_i x}{A_i d} \quad \forall \alpha \geq 0$$

MASSIMO PASSO

$$\bar{\alpha} = \min \left\{ \frac{b_i - A_i x}{A_i d} : A_i d > 0 \right\}$$

Quindi

$$d \text{ AMMISSIBILE} \Leftrightarrow A_i d \leq 0 \quad \forall i : A_i x = b_i$$

$$I(x) = \{i : A_i x = b_i\} \text{ VINCOLI ATTIVI}$$

$$\{d \in \mathbb{R}^n : A_{I(x)} d \leq 0\}$$

CONO POLIEDRICO
delle DIREZIONI AMMISSIBILI

DIREZIONE DI CRESCITA d è di CRESCITA se $cd > 0$, ovvero se mi muovo da x a $x + \alpha d$ con $\alpha d > 0$
 allora $cx = c(x + \alpha d) = cx + \alpha(cd) > cx$

TEOREMA $\nexists d$ DIREZIONE AMMISSIBILE di CRESCITA per $x \iff x$ è OTTIMO

DIMOSTRAZIONE • $\nexists d$ DIREZIONE AMMISSIBILE di CRESCITA $\Rightarrow x$ è OTTIMO

Per assurdo $\nexists d$ AMMISSIBILI di CRESCITA e x non è OTTIMO

$x \in P$ e $x^* \in P$, $cx^* > cx$ ma allora $d = x^* - x$ è AMMISSIBILE per x per CONVESSITÀ

quindi $x + 1d = x + d = x^*$, ma allora $cx^* > cx \equiv cx^* - cx > 0 \equiv c(x^* - x) > 0 = cd > 0$ ■

• $\exists d$ DIREZIONE AMMISSIBILE di CRESCITA $\Rightarrow x$ NON è OTTIMO

$x + \alpha d \in P \quad \exists \alpha > 0 \quad c(x + \alpha d) = cx + \alpha(cd) > cx$ ■

TRASFORMAZIONE CONO FINITAMENTE GENERATO $\longrightarrow$ CONO POLIEDRICO

$$\begin{aligned} \{x \in \mathbb{R}^n : Ax \leq b\} &\equiv \text{cono}(\{v_1, v_2, \dots, v_h\}) \\ &\{x = \sum_{i=1}^h b_i v_i : b_i \geq 0 \quad i=1 \dots h\} \\ x &= V\beta \quad \beta \geq 0 \quad V = [v_1, v_2, \dots, v_h] \quad \beta = \begin{bmatrix} \beta_1 \\ \vdots \\ \beta_h \end{bmatrix} \\ \begin{bmatrix} I & -V \\ 0 & -I \end{bmatrix} \begin{bmatrix} x \\ \beta \end{bmatrix} &\leq \begin{bmatrix} 0 \\ 0 \end{bmatrix} \\ &\downarrow \text{Rimozione di } = \\ \begin{bmatrix} I & -V \\ -I & V \\ 0 & -I \end{bmatrix} \begin{bmatrix} x \\ \beta \end{bmatrix} &\leq \begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix} \end{aligned}$$

PROBLEMA ! Il processo inverso (vincoli $\rightsquigarrow$ DIREZIONI v) può essere fatto ma potrebbe portare da un numero finito di vincoli K ad un numero ESPONENZIALE di vertici 2^K . Gli unici casi in cui questo non succede è con i

CONI SIMPLICIALI

$$A_{I(x)} d \leq 0 \quad \text{con} \quad \begin{cases} |I(x)| = n & (\text{MATRICE QUADRATA } n \times n) \\ \det(A_{I(x)}) \neq 0 & (\text{MATRICE INVERTIBILE}) \end{cases}$$

Un insieme di indici $I(x) = B$ per cui otteniamo un CONO SIMPLICIALE è chiamato BASE
 $\{d \in \mathbb{R}^n : A_B d \leq 0\}$

TRASFORMAZIONE CONO POLIEDRICO $\longrightarrow$ CONO FINITAMENTE GENERATO (CONO SIMPLICIALE)

$$\{d \in \mathbb{R}^n : A_B d \leq 0\}$$

$\downarrow$

$$\{d \in \mathbb{R}^n : A_B d = -\alpha, \alpha \geq 0\}$$

$\downarrow$

! A è INVERTIBILE perché il cono è SIMPLICIALE

$$\{d \in \mathbb{R}^n : d = -\underbrace{A_B^{-1}}_{v_1, v_2, \dots, v_m} \alpha, \alpha \geq 0\} \longrightarrow d = v_1 \alpha_1 + v_2 \alpha_2 + \dots + v_m \alpha_m$$

NOTA CALCOLO dell'INVERSA

$$A_B = \begin{bmatrix} a & b \\ c & d \end{bmatrix} \quad A_B^{-1} = \frac{1}{\det(A_B)} \begin{bmatrix} d & -b \\ -c & a \end{bmatrix}$$

ESEMPIO

$$\max 5x_1 + 2x_2$$

$$c = [5, 2]$$

$$(1) \quad x_1 \leq 4$$

$$(2) \quad x_2 \leq 7$$

$$(3) \quad 2x_1 + x_2 \leq 9$$

$$(4) \quad -x_1 \leq 0$$

$$(5) \quad -x_2 \leq 0$$

$$A =$$

$$\begin{bmatrix} 1 & 0 \\ 0 & 1 \\ 2 & 1 \\ -1 & 0 \\ 0 & -1 \end{bmatrix}$$

• Partiamo da $x = [1, 7]$, i vincoli attivi sono $I(x) = \{2, 3\}$

perché (2) $x_2 = 7 \rightarrow 7 = 7$ e (3) $2x_1 + x_2 = 9 \rightarrow 2 \cdot 1 + 7 = 9$

• $A_{I(x)} = \begin{bmatrix} 0 & 1 \\ 2 & 1 \end{bmatrix}$ è $n \times n$ ed è INVERTIBILE ($\det = -2 \neq 0$)

quindi $I(x)$ è una base B

$$A_B^{-1} = \begin{bmatrix} -\frac{1}{2} & \frac{1}{2} \\ 1 & 0 \end{bmatrix} \quad d = -A_B^{-1} \alpha = \begin{bmatrix} \frac{1}{2} & -\frac{1}{2} \\ -1 & 0 \end{bmatrix} \alpha$$

• Verifico se x è ottimo, ovvero controllo se v_1 e v_2 sono

$$\text{direzioni di crescita} \quad c \cdot v_1 = [5, 2] \begin{bmatrix} \frac{1}{2} \\ -1 \end{bmatrix} = \frac{1}{2} \quad c \cdot v_2 = [5, 2] \begin{bmatrix} -\frac{1}{2} \\ 0 \end{bmatrix} = -\frac{5}{2}$$

quindi ci dobbiamo spostare seguendo la retta $x + \alpha v_1$

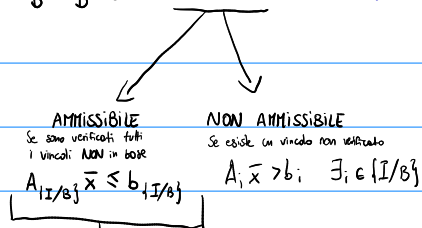
E se i VINCOLI ATTIVI NON corrispondono ad una BASE?

Partiamo da una FACCIA $P_S = \{x \in \mathbb{R}^n : A_i x = b_i, i \in S\}$ con $I = \{1, 2, \dots, m\}$ e $S \subset I$
 $A_i x \leq b_i, i \in I/S$

Quando una faccia è MINIMA? Quando è un PUNTO, ovvero quando $A_S x = b_S$ ammette SOLUZIONE UNICA

Questo succede quando A_S è QUADRATA e INVERTIBILE, ovvero quando S è una BASE $S=B$

Quindi $\bar{x} = A_B^{-1} b_B$ è SOLUZIONE PRIMALE di BASE associata alla BASE B



NOTA

- Non tutte le basi B corrispondono ad un vertice del poliedro
- Un vertice può essere collegato a PIÙ di UNA BASE



L'insieme dei VINCOLI ATTIVI di un vertice CONTIENE
la base ma non vuol dire che lo sia esattamente

$I(\bar{x}) = B$ NON DEGENERE $\forall i \in N: A_i \bar{x} < b_i$
 $I(\bar{x}) \supsetneq B$ DEGENERE $\exists i \in N: A_i \bar{x} = b_i$

ALGORITMO del SIMPLESSO PRIMALE

Simplex-Primale $(A, b, c, \overset{\text{Base PRIMALE AMMISSIBILE}}{B})$

For (stato = ""; j)

$\bar{x} = A_B^{-1} b_B$ → Soluzione associata

$\bar{y} = [\bar{y}_B, \bar{y}_N] = [c A_B^{-1}, \emptyset]$
↓
OPPOSTO
DIREZIONI AMMISSIBILI

if $(\bar{y}_B \geq 0)$ then { stato = "ottimo"; break; } → Non ho direzioni ammissibili di crescita

$h = \min \{ i \in B : \bar{y}_i < 0 \}$; → Prendo il primo INDICE di VINCOLO in BASE

$\varepsilon = -A_B^{-1} u_{B(h)}$; → Estraggo la sua DIREZIONE, $u_{B(h)} = \begin{bmatrix} 0 \\ 1 \\ 0 \end{bmatrix} \rightarrow h$

if $(A_N \varepsilon \leq 0)$ then { stato = "P. illimitato"; break; } → Tutte le direzioni ammissibili di crescita sono LIMITATE

$\bar{\lambda} = \min \{ \lambda_i = \frac{b_i - A_i \bar{x}}{A_i \varepsilon}, A_i \varepsilon > 0, i \in N \}$; → Calcolo il passo massimo lungo ε

$K = \min \{ i \in N : \lambda_i = \bar{\lambda} \}$; → Prendo l'indice minimo dei vincoli attivi che ho trovato con $\bar{\lambda}$

$B = B \cup \{K\} \setminus \{h\}$; → PIVOT: aggiorno la base

Regole Anticiclo
di BLEND

CORRETTEZZA

- Ogni iterazione è AMMISSIBILE perché partiamo da $\bar{x}$ ottenuto da B ammissibile e facciamo il passo $\bar{x} + \bar{\lambda} \varepsilon$ che è ammissibile oppure un passo $\emptyset$ se ε non è ammissibile

- Ogni nuova base B' è ancora una base

- Se sostituiamo un solo elemento, rimane $n \times n$

- La riga sostituita è LIN. INDIP. dalle altre e quindi A_B rimane INVERTIBILE

DIMOSTRAZIONE Se la nuova riga A_K è LIN. DIP. allora $\beta A_{B/h} = A_K \varepsilon$

con $\varepsilon = A_B^{-1} u_{B(h)}$. Ma allora preso un $i \in B \setminus \{h\}$, $A_i \varepsilon = A_i A_B^{-1} u_{B(h)} = 0$

MA $A_i \varepsilon > 0$ per definizione



TERMINAZIONE Se non esistessero BASI DEGENERI non ci sarebbero problemi perché ogni volta che facciamo un passo questo è STRETTAMENTE MAGGIORE. Ma noi le abbiamo...

REGOLA ANTICICLO di BLEND: prendere il vincolo di indice minore

Ricerca di una BASE AMMISSIBILE - GREEDY

Parto da una matrice A con $\text{rango}(A)=n$. Prendo la prima riga e la inserisco nella base B . Poi iterativamente per ogni riga successiva verifico se questa è L.V. DIP della base: in caso positivo la scarto, altrimenti la aggiungo.

Come capisco se la base B è AMMISSIBILE? $H = \{i \in N : A_i \bar{x} \leq b\}$ $N = I/B$
 $J = \{i \in N : A_i \bar{x} > b\}$ $\bar{x} = A_B^{-1} b_B$

$\rightarrow H = N \Rightarrow \bar{x}$ AMMISSIBILE
 $\rightarrow J \neq \emptyset \Rightarrow \bar{x}$ NON AMMISSIBILE

Definisco un nuovo problema di PL

$$\max x - u v_j$$

$$\begin{bmatrix} A_B & \emptyset \\ A_H & \emptyset \\ A_J & -I_J \\ \emptyset & -I_J \end{bmatrix} \begin{bmatrix} x \\ v_j \end{bmatrix} \leq \begin{bmatrix} b_B \\ b_H \\ b_J \\ \emptyset \end{bmatrix}$$

In pratica permetto a B di cambiare per un certo v_j ma pagando un costo u

La cui base è $B = B \cup J$



Uso il SIMPLEXO PRIMALE sul nuovo problema e ottengo (x^*, v_j^*)

$v_j^* \neq 0$ $v_j^* = 0$
 Il primo problema è **VUOTO** B è ammissibile per il primo problema

DUALITÀ

Ho un sistema con dei vincoli $Ax \leq b$. Per essere soluzione deve rispettarli tutti e, in particolare, deve rispettare quelli in base

$$\forall x: Ax \leq b \quad A_B x \leq b_B$$

e a questo punto ho anche un $\bar{y}_B = c A_B^{-1} \geq 0$ perché $\bar{x}$ è OTTIMA

$$\bar{y}_B (A_B) x \leq \bar{y}_B b_B \quad (\text{Moltiplico entrambi i termini per } \bar{y}_B)$$

$$c A_B^{-1} A_B x \leq c A_B^{-1} b_B$$

$$c x \leq c \bar{x}$$

Qualunque soluzione ammissibile x ha un valore NON MIGLIORE della soluzione ottima.

Dati m vincoli $Ax \leq b$, $A \in \mathbb{R}^{m \times n}$, $b \in \mathbb{R}^m$ e un vettore di numeri $y \geq 0$, $y \in \mathbb{R}_+^m$ posso ottenere un singolo vincolo $y(Ax) \leq yb \rightarrow (yA)x \leq (yb)$

Se poniamo $(yA) = c \rightarrow cx \leq yb \rightarrow \bar{y}b \geq z(P)$ VALUTAZIONE SUPERIORE
 $c\bar{x} \leq z(P)$ VALUTAZIONE INFERIORE

TEOREMA DEBOLE DELLA DUALITÀ

$$\left. \begin{array}{l} \forall \bar{x} \text{ AMMISSIBILE per (P)} \quad c\bar{x} \leq z(P) = c x^* \\ \forall \bar{y} \text{ AMMISSIBILE per (D)} \quad \bar{y}b \geq z(P) \end{array} \right\} c\bar{x} \leq \bar{y}b$$

$$z(P) \leq z(D)$$

Questo ci permette di capire quando (P) e (D) sono VUOTI

$$z(D) = -\infty \Rightarrow (P) \text{ VUOTO}$$

$$z(P) = -\infty$$

$$z(P) = +\infty \Rightarrow (D) \text{ VUOTO}$$

$$z(D) = +\infty$$

P \ D	OTTIMO	VUOTO	ILLIMIT
OTTIMO	✓	✗	✗
VUOTO	✗	✓	✓
ILLIMIT	✗	✓	✗

TEOREMA FORTE DELLA DUALITÀ

Se $\exists \bar{x}$ AMMISSIBILE per (P) e $\bar{y}$ AMMISSIBILE per (D) allora

$$-\infty < z(P) = z(D) < +\infty$$

INTUITIVAMENTE $\bar{x} = A_B^{-1} b_B$ $\bar{y} = [\bar{y}_B, \bar{y}_N] = [c A_B^{-1}, 0]$ $\bar{y} \geq 0$

$$yA = [\bar{y}_B, \bar{y}_N] \begin{bmatrix} A_B \\ A_N \end{bmatrix} = \bar{y}_B A_B + \bar{y}_N A_N = \bar{y}_B A_B + 0 = c A_B^{-1} A_B = c$$

$$y b = [c A_B^{-1}, 0] \begin{bmatrix} b_B \\ b_N \end{bmatrix} = c A_B^{-1} b_B + 0 b_N = c \bar{x}$$

$$c \bar{x} \leq z(P) \leq z(D) \leq \bar{y} b \quad (\text{TEOREMA DEBOLE DELLA DUALITÀ})$$

$$c \bar{x} = \bar{y} b \Rightarrow z(P) = z(D)$$

DIMOSTRAZIONE L'algoritmo del SIMPLESSO funziona

ESEMPIO: (P) e (D) entrambi vuoti

$$(P) \max \left\{ \begin{bmatrix} c \\ 0, 1 \end{bmatrix} \begin{bmatrix} x \\ x_1 \\ x_2 \end{bmatrix} : \begin{bmatrix} 1 & 0 \\ -1 & 0 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} \leq \begin{bmatrix} b \\ -1 \\ -1 \end{bmatrix} \right\} \quad (D) \min \left\{ \begin{bmatrix} y_1 & y_2 \end{bmatrix} \begin{bmatrix} -1 \\ -1 \end{bmatrix} : \begin{bmatrix} y_1 & y_2 \end{bmatrix} \begin{bmatrix} 1 & 0 \\ -1 & 0 \end{bmatrix} = \begin{bmatrix} c \\ 0 & 1 \end{bmatrix} \right\}$$

CONDIZIONE DEGLI SCARTI

Se $\bar{x}$ e $\bar{y}$ ammettono entrambi soluzioni ammissibili allora

COMPLEMENTARI

$$c x^* = y^* b$$

per qualsiasi x^* OTTIMA per (P)
 y^* OTTIMA per (D)

$$(y^* A) x^* = y^* b$$

$$y^* (b - A x^*) = 0$$

↓
PRODOTTO
SCALARE

$$\sum_{i=1}^m y_i^* (b_i - A_i x^*) = 0$$

$$y_i^* \geq 0, \quad b_i - A_i x^* \geq 0$$

$$y_i^* (b_i - A_i x^*) = 0 \quad \forall i = 1 \dots m$$

$$y_i^* > 0$$

$$\Downarrow$$

$$A_i x^* = b_i$$

$$A_i x^* < b_i$$

$$\Downarrow$$

$$y_i^* = 0$$

DIMOSTRAZIONE

$$\bar{y} (b - A \bar{x}) = 0 \rightarrow [c A_B^{-1}, 0] \left(\begin{bmatrix} b_B \\ b_N \end{bmatrix} - \begin{bmatrix} A_B \\ A_N \end{bmatrix} A_B^{-1} b_B \right) = 0$$

$$c A_B^{-1} (b_B - A_B A_B^{-1} b_B) + 0 (b_N - A_N A_B^{-1} b_B) = 0 + 0 = 0$$

NOTA Analogamente alle soluzioni del problema primale, quelle del problema duale possono essere riferite ad una BASE B

- AMMISSIBILE $\bar{y} \geq 0$ $\bar{y} = [\bar{y}_B, \bar{y}_N] = [c A_B^{-1}, 0]$
- NON AMMISSIBILE $\exists i \in B : \bar{y}_i < 0$

NOTA Analogamente al caso primale, una BASE B duale può essere

- DEGENERE $\exists i \in B : \bar{y}_i = 0$
- NON DEGENERE $\forall i \in B : \bar{y}_i \neq 0$

Snake Eyes 

Quando abbiamo una base (sia primale che duale) degenera, entrambi i termini dello scarto complementare sono 0 e vuol dire che le soluzioni trovate per (P) e (D) non sono uniche.

$$i \in B \quad b_i - A_i \bar{x} = 0 \quad \bar{y}_i = 0$$

$$i \in N \quad b_i - A_i \bar{x} = 0 \quad \bar{y}_i = 0$$

ESEMPIO: PRIMALE e DUALE

$$\max 5x_1 + 2x_2$$

$$c = [5, 2]$$

$$x_1 \leq 4$$

$$x_2 \leq 7$$

$$(P) \quad 2x_1 + x_2 \leq 9$$

$$-x_1 \leq 0$$

$$-x_2 \leq 0$$

$$b = \begin{bmatrix} 4 \\ 7 \\ 9 \\ 0 \\ 0 \end{bmatrix}$$

$$A = \begin{bmatrix} 1 & 0 \\ 0 & 1 \\ 2 & 1 \\ -1 & 0 \\ 0 & -1 \end{bmatrix}$$

$$y = [y_1, y_2, y_3, y_4, y_5] \quad yA = c \quad y_1 + 2y_3 - y_4 = 5$$

$$y \geq 0$$

$$y_2 + y_3 - y_5 = 2$$

$$y_1, y_2, y_3, y_4, y_5 \geq 0$$

(D)

$$\min yb \quad \min 4y_1 + 7y_2 + 9y_3$$

Partiamo da una soluzione PRIMALE $x = [4, 1]$. Vogliamo capire se è OTTIMA

$I(x) = \{1, 3\} \rightarrow$ Le condizioni 2, 4 e 5 hanno uno SCARTO

Quindi per il TEOREMA degli SCARTI COMPLEMENTARI $y_2 = y_4 = y_5 = \emptyset$ e quindi in (D) rimane

$$\begin{aligned} y_1 + 2y_3 &= 5 \\ y_3 &= 2 \end{aligned} \rightarrow [\bar{y}_1, \bar{y}_3] = [1, 2] > \emptyset$$

è l'unica soluzione ottima del DUALE. Vale il viceversa per il primale.

Questo funziona perché la base $\{1, 3\}$ è NON DEGENERE e AMMISSIBILE se per (P) e (D)

Se ad esempio prendiamo $x = [1, 7]$ abbiamo $I(x) = \{2, 3\}$

$$\bar{y}_1 = \bar{y}_4 = \bar{y}_5 = \emptyset \rightarrow 2y_3 = 5 \quad [\bar{y}_2, \bar{y}_3] = [-\frac{1}{2}, \frac{5}{2}]$$

$$y_2 + y_3 = 2 \quad \bar{y} \text{ NON AMMISSIBILE } (\bar{y}_2 < 0)$$

$\downarrow$
X NON OTTIMA

QUINDI...

BASE PRIMALE NON DEGENERE	$\rightarrow$ Soluzione DUALE UNICA OTTIMA
DEGENERE	$\rightarrow$ Potrebbero esserci più soluzioni duali ottime
DUALE NON DEGENERE	$\rightarrow$ Soluzione PRIMALE UNICA OTTIMA
DEGENERE	$\rightarrow$ Potrebbero esserci più soluzioni primali ottime

COSTI OMBRA

Consideriamo un problema di PL: $Ax \leq b$ $A_i x \leq b_i \quad i = 1 \dots m$ e ci sommiamo ε , che rappresenta una certa quantità "extra" di materiale che possiamo acquistare.

$$(P_\varepsilon) \max \{cx : Ax \leq b + \varepsilon\}$$

$$(D_\varepsilon) \min \{Y(b + \varepsilon) : YA = C, Y \geq 0\}$$

Consideriamo il caso in cui non compriamo niente, $\varepsilon = 0$, e otteniamo le soluzioni ottime

$$(P_0) \quad x^* \quad Ax^* \leq b$$

$$(D_0) \quad y^* \quad y^* A = C, y^* \geq 0$$

Notiamo che la soluzione DUALE non dipende da ε , quindi

$$y^*(b + \varepsilon) \leq z(D_\varepsilon) = z(P_\varepsilon)$$

$$y^* b + y^* \varepsilon$$

$$cx^* + y^* \varepsilon \quad \text{con} \quad cx^* = z(P) = z(D)$$

$$\text{quindi} \quad z(P_\varepsilon) \leq z(P) + y^* \varepsilon \quad \text{con} \quad y^* \varepsilon = \sum_{i=1}^m y_i^* \varepsilon_i$$

quindi le variabili duali y_i^* rappresenta quanti soldi in più facciamo comprando più risorse i

$\downarrow$
VALORE UNITARIO

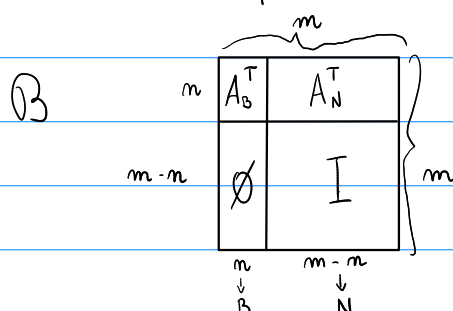
ALGORITMO del SIMPLESSO DUALE

Partiamo da una base B DUALE AMMISSIBILE, quindi $\bar{y} = [\bar{y}_B, \bar{y}_N] = [c A_B^{-1}, 0]$ $\bar{y} \geq 0$ $\bar{y} A = c$ e il PL corrispondente $\min \{y b : y A = c, y \geq 0\}$. Lo trasformo nella forma di un problema primale (CANONICA):

$$-\max \{ \bar{c} y^T : \bar{A} y^T \leq \bar{b} \}$$

$$\bar{c} = -b^T \quad \bar{A} = \begin{bmatrix} A^T & 0 \\ -A^T & I \end{bmatrix} \quad \bar{b} = \begin{bmatrix} c^T \\ -c^T \end{bmatrix}$$

In questo caso $A \in \mathbb{R}^{m \times (2n+m)}$ quindi la base B corrispondente deve essere quadrata $m \times m$, ma in realtà le basi B OTTIME sono attaccate ai vertici di entrambi i problemi.



È DIAGONALE a BLOCCHI e per essere invertibile I e A_B^T devono esserlo

Simplexso-DUALE $(A, b, c, \bar{B})$ Base DUALE AMMISSIBILE

for (stato = " "; ;)

$\bar{x} = A_B^{-1} b_B$ $\rightarrow$ Soluzione primale di base

$\bar{y} = [\bar{y}_B, \bar{y}_N] = [c A_B^{-1}, 0]$ $\rightarrow$ Soluzione duale di base

if ($A_N \bar{x} \leq b_N$) then { stato = "ottimo"; break; } $\rightarrow$ Controllo se la soluzione primale di base è ammissibile

$K = \min \{ i \in N : A_i \bar{x} > b_i \}$ $\rightarrow$ Prendo il primo vincolo VIOLATO K

$\eta_B = A_K A_B^{-1}$ $\rightarrow$ Estraggo il suo GRADIENTE (direzione ammissibile di DECRESCITA)

if ($\eta_B \leq 0$) then { stato = "P. vuoto"; break; } $\rightarrow$ (D) INF. ILL = (P) VUOTO

$\bar{\theta} = \min \{ \theta_i = \frac{\bar{y}_i}{\eta_i} : \eta_i > 0, i \in B \}$

$h = \min \{ i \in B : \theta_i = \bar{\theta} \}$

$B = B \cup \{K\} \setminus \{h\}$ $\rightarrow$ PIVOT = aggiornare la base

FUNZIONA PERCHÉ

! TUTTE LE COPPIE di SOLUZIONI di BASE

RISPETTANO SEMPRE LE CONDIZIONI

DEGLI SCARTI COMPLEMENTARI

$\eta_B = A_K \begin{bmatrix} A_B^{-1} \\ 0 \end{bmatrix} = \begin{bmatrix} A_K A_B^{-1} \\ 0 \end{bmatrix}$ $\rightarrow$ è una DIREZIONE nello spazio duale

Identifichiamo la DIREZIONE di DECRESCITA: $\bar{y} + \theta d$ AMMISSIBILE

DECRESCITA

Peró $\bar{y} \in \mathbb{R}^m$ e quindi $d \in \mathbb{R}^m$. EPPURE! $\eta \in \mathbb{R}^m$ quindi

$$d_i = \begin{cases} -\eta_i & i \in B \\ 1 & i = K \\ 0 & \text{ALTRIMENTI} \end{cases} \quad i = 1 \dots m$$

Quindi, mostriamo che é di DECRESCITA

$$db = [-\eta_B, 1, 0] \begin{bmatrix} b_B \\ b_K \\ \dots \end{bmatrix} = -\eta_B b_B + b_K + 0 = -A_K A_B^{-1} b_B + b_K =$$

$$= b_K - A_K (A_B^{-1} b_B) =$$

$$= \underbrace{b_K - A_K \bar{x}}_{\downarrow}$$

Dato che $K = \min \{i \in N : A_i \bar{x} > b_i\}$

$$b_K - A_K \bar{x} < 0 \Rightarrow \text{DECRESCITA}$$

AMMISSIBILITA'

Per l'AMMISSIBILITA':

UGUAGLIANZA	$(\bar{y} + \theta d)A = c \quad dA = [-\eta_B, 1, 0] \begin{bmatrix} A_B \\ A_K \\ \dots \end{bmatrix} = -\eta_B A_B + A_K + 0 = -A_K A_B^{-1} A_B + A_K =$ $\bar{y}A + \theta dA = c \quad \quad \quad = -A_K + A_K = 0$ $\theta dA = 0$ $dA = 0$
-------------	--

DISUGUAGLIANZA	$\bar{y}_i + \theta d_i \geq 0 \quad i = 1 \dots m$ $\downarrow \bar{y} \geq 0$ $\begin{cases} \bar{y}_i - \theta \eta_i \geq 0 & i = B \rightarrow \text{Dipende dal segno di } \eta \\ \bar{y}_i + \theta \geq 0 & i = K \rightarrow 0 \text{ uio, } \bar{y} \geq 0 \quad \theta \geq 0 \\ \bar{y}_i + \theta \cdot 0 \geq 0 & \text{ALTRIMENTI} \rightarrow 0 \text{ uio, } \bar{y} \geq 0 \end{cases}$	$\eta_i \leq 0 \rightarrow \text{Qualsiasi } \theta$ $\eta_i > 0 \rightarrow \begin{cases} \bar{y}_i - \theta \eta_i \geq 0 \\ \bar{y}_i \geq \theta \eta_i \end{cases}$
----------------	--	---

$$\theta_i \leq \frac{\bar{y}_i}{\eta_i}$$

PASSO CHE AZZERA LE VARIABILI

↓
Prendo la più piccola perché se vado sotto 0 non é più AMMISSIBILE

$$\bar{y}_i = 0 \Rightarrow \theta = 0$$

PASSO DEGENERE

CORRETTEZZA e TERMINAZIONE È corretto e termina perché in realtà è uguale al semplice PRIMALE

TROVARE BASE DUALE AMMISSIBILE Come per la base primale, costruisco un nuovo problema ed eseguo il SEMPLICE DUALE

$$(DA) \min \{ yb + wM : yA + w = c, y \geq 0, w \geq 0 \} \quad (PA) \max \{ cx : Ax \leq b, x \leq M \}$$



$$\text{BASE } B = \{ m+1, \dots, m+n \}$$

AMMISSIBILE perché la soluzione è $[\bar{y}, \bar{w}] = [0, c]$ ($c \geq 0$, in caso contrario bastava moltiplicare per -1)

Eseguo SEMPLICE DUALE

→ (D) INF. ILL. $\Rightarrow$ (DA) INF. ILL.

→ (D) NON INF. ILL. $\Rightarrow \exists M. [y^*, w^*]$

$w^* = \emptyset \Rightarrow y^*$ OTTIMA per (D)

$w^* \neq \emptyset \Rightarrow$ (D) Non ha sol. AMM.

VARIAZIONE di C

Se c diventa c' , la matrice di base A_B rimane PRIMALE AMMISSIBILE e $\bar{x}$ rimane la soluzione corrispondente.

La SOLUZIONE DUALE cambia e potrebbe essere necessario ($\bar{y}_B \geq 0$) ottimizzare con il semplice primale.

VARIAZIONE di b

Se una componente k di b cambia in b'_k , la SOLUZIONE DUALE NON CAMBIA

Quella PRIMALE:

$k \in N \rightarrow A_k \bar{x} \leq b'_k \rightarrow A_B$ rimane OTTIMA

ALTRIMENTI $\rightarrow$ Applico il SEMPLICE DUALE

$k \in B \rightarrow \bar{x}' = A_B^{-1} b'_B, A_N \bar{x}' \leq b_N \rightarrow B$ rimane OTTIMA

ALTRIMENTI $\rightarrow$ Applico il SEMPLICE DUALE

AGGIUNTA VINCOLO in (P) = VARIABILE in (D)

Aggiungendo il vincolo $A_{m+1} x \leq b_{m+1}$ $A_{m+1} \bar{x} \leq b_{m+1}$ allora A_B rimane OTTIMA

Altrimenti applico il DUALE conoscendo già l'indice $m+1$ della variabile duale che entra in base

AGGIUNTA VARIABILE in (P) = VINCOLO in (D)

Aggiungendo la variabile x_{m+1} con colonna A_{m+1} e costo c_{m+1} otteniamo $A' = \begin{bmatrix} A & A_{m+1} \\ \emptyset & -1 \end{bmatrix}$ e $A'_B = \begin{bmatrix} A_B & A_{m+1} \\ \emptyset & -1 \end{bmatrix}$
associata alla base $B' = B \cup \{m+1\}$ con $N' = N$
VINCOLO DI NON NEGATIVITÀ

Le nuove soluzioni sono:

$$x' = [\bar{x}, x'_{m+1}] = [\bar{x}, 0] \rightarrow \text{AMMISSIBILE}$$

$$y' = [y'_B, y'_N] \text{ con } y'_N = \emptyset \text{ e } y'_B = [\bar{y}_B, y'_{m+1}] \text{ con } y'_{m+1} = \bar{y}_B A_B^{-1} - c_{m+1}$$

≥ 0 B' AMMISSIBILE

Altrimenti applico il SEMPLICE PRIMALE

PROGRAMMAZIONE LINEARE INTERA

Aggiungiamo il vincolo di INTEGRALITÀ (in pratica DISCRETIZZIAMO) e perdiamo la CONVESSITÀ.

ESEMPIO: problema dello ZAINO

$$\max 2x_1 + 8x_2 + 4x_3 + 7x_4 + x_5 \quad (\text{VALORE})$$

Nota: possiamo prendere un oggetto una sola volta

$$3x_1 + 3x_2 + 2x_3 + 4x_4 + 2x_5 \leq 8 \quad (\text{PESO})$$

$$x_1, x_2, x_3, x_4, x_5 \in \{0, 1\}$$

RILASSIAMO il problema! $\hookrightarrow [0, 1]$, discreto, NO INTEGRALITÀ

Calcoliamo il costo unitario e riordiniamo le variabili

$$\frac{2}{3} \quad \frac{8}{3} \quad \frac{4}{2} \quad \frac{7}{4} \quad \frac{1}{2} \longrightarrow x_2, x_3, x_4, x_1, x_5$$

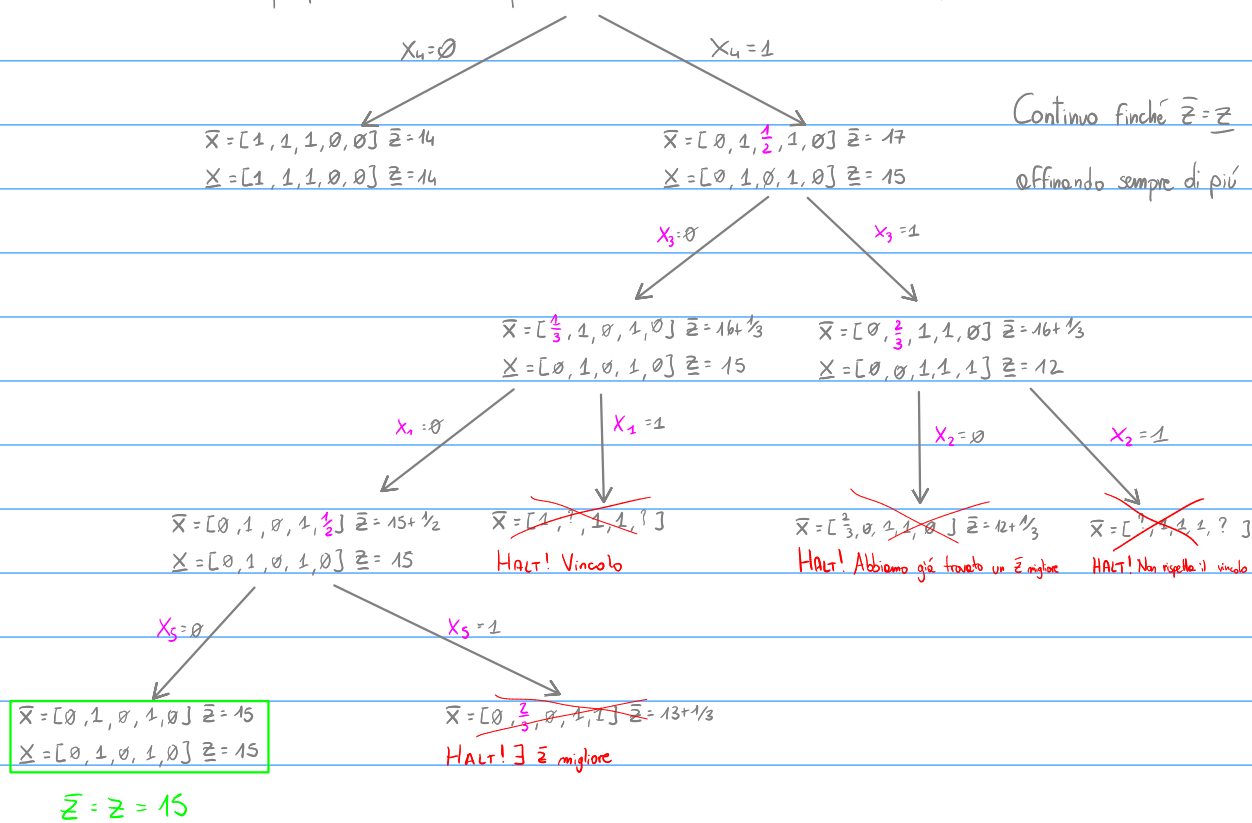
Infiliamo nello zaino in ordine finché c'è spazio e otteniamo un

$$\text{UPPER BOUND } \bar{x} = [0, 1, 1, \frac{3}{4}, 0] \quad \bar{z} = 17 + \frac{1}{4}$$

Per ottenere il LOWER BOUND azzeriamo la variabile che ha valore discreto

$$x = [1, 1, 1, 0, 0] \quad z = 14$$

Da qui possiamo dividere il problema in due istanze su x_4 (0 e 1)



META-ALGORITMO BRANCH & Bound

Dobbiamo poter RILASSARE un problema $(P) \max \{c(x) : x \in X\} \rightsquigarrow (\bar{P}) \max \{\bar{c}(x) : x \in \bar{X}\}$. Questo significa che tutte le soluzioni del problema originale sono anche in quello rilassato $\bar{X} \supseteq X$ e $\bar{c}(x) \geq c(x) \forall x \in X$

Risolvere il rilassamento ci garantisce una VALUTAZIONE SUPERIORE x^* di (P) : $\bar{c}(x^*) = \bar{z} \geq z(P)$

Nel caso in cui $x^* \in X$ (è soluzione di (P)) ci dà anche una VALUTAZIONE INFERIORE: $c(x^*) \leq z(P)$

Se $c(x^*) = \bar{c}(x^*)$ abbiamo trovato la SOLUZIONE al problema (P)

Le EURISTICHE prendono il problema e cercano di dare VELOCEMENTE una SOLUZIONE AMMISSIBILE, senza garantire che sia ottima.

Se la soluzione trovata dal RILASSAMENTO non è ottima, si applica DIVIDE et IMPERA, ovvero dato un problema di regione ammissibile X lo suddivido in $(P_i) \max \{c(x) : x \in X_i\}$ con $X_i \subset X$, $i=1 \dots K$ e $\bigcup_{i=1}^K X_i = X$. A questo punto vale:

$$\max \{z(P_i) : i=1, \dots, K\} \leq z(P) \leq \max \{\bar{z}(P_i) : i=1, \dots, K\}$$

$$z = B \& B(P, \varepsilon)$$

$Q = \{(P)\};$

$z = -\infty;$

do {

$(P') = \text{Next}(Q);$

$Q = Q \setminus \{(P')\};$

→ Prendo un sotto problema

$(\bar{z}, z) = \text{Rilassamento}(P');$

if $(z > z)$ then $z = z$; → Ho trovato una soluzione migliore

if $(\bar{z} \leq z + \varepsilon)$ then continue; → La soluzione è peggiore (con tolleranza ε) e termino questo NODO

$z = \text{Euristica}(P');$

if $(z > z)$ then $z = z$;

if $(\bar{z} \leq z + \varepsilon)$ then continue;

$Q = Q \cup \text{Branch}(P');$

} While $(Q \neq \emptyset);$

TERMINAZIONE Ci sono un numero finito di possibili decisioni sulle variabili e quindi di nodi nell'albero di branching.

CORRETTEZZA In ogni momento abbiamo una VALUTAZIONE SUPERIORE sull'ottimo del problema, questa continua a scendere finché, alla fine dell'algoritmo, diventa uguale a quella INFERIORE.

VALUTAZIONE SUPERIORE

Partiamo definendo Q come l'insieme dei nodi ancora da espandere e Q'' quello dei nodi MORTI

$$z(p) \leq \max \left\{ \begin{array}{l} \max \{ \bar{z}(p') : p' \in Q \} \\ \max \{ \bar{z}(p'') : p'' \in Q'' \} \end{array} \right\}$$

Questo è vero perché $X = \left(\bigcup_{p' \in Q} X' \right) \cup \left(\bigcup_{p'' \in Q''} X'' \right)$

Ora definisco Q' come l'insieme dei **PREDECESSORI IMMEDIATI** dei nodi in Q

e vedo che $\bigcup_{p'' \in Q''} X'' \supseteq \bigcup_{p' \in Q'} X'$, quindi vale

$$z(p) \leq \max \left\{ \begin{array}{l} \max \{ \bar{z}(p') : p' \in Q' \} \\ z + \varepsilon \end{array} \right\}$$

TEOREMA L'algoritmo è corretto, $z \leq z(p) \leq z + \varepsilon$, perché al termine $Q = \emptyset$ e quindi $Q' = \emptyset$ e quindi $z(p) \leq \max \left\{ \begin{array}{l} \max \{ \emptyset \} \\ z + \varepsilon \end{array} \right\} \leq z + \varepsilon$

COME FUNZIONA per la PLI

RILASSAMENTO toglie il vincolo di integralità e diventa un problema di PL risolvibile con il SIMPLESSO

EURISTICA la ottengo risolvendo un problema di PL

BRANCHING se ottengo un $\bar{x} \notin \mathbb{Z}^n$, ovvero $\exists i : \bar{x}_i \notin \mathbb{Z}$, divido come segue:

$$\bullet X_i \leq \lfloor \bar{x}_i \rfloor$$

$$\bullet X_i \geq \lceil \bar{x}_i \rceil$$

dato che viene aggiunto un **VINCOLO** uso il **SIMPLESSO DUALE**

che in questi casi è molto più veloce del **PRIMALE**

REGOLE di BRANCHING

Devono rendere INAMMISSIBILE la soluzione del Rilassamento per garantire il miglioramento della VALUTAZIONE SUPERIORE.

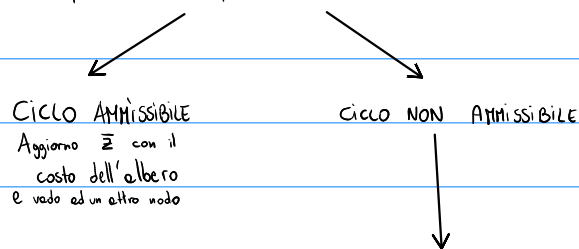
$$\begin{array}{l} (P') \max \{c(x) : x \in X'\} \quad X' \subseteq X \longrightarrow x^* \in X' \quad \begin{array}{l} \nearrow x^* \notin X'_0 \\ \searrow x^* \notin X'_1 \end{array} \\ (\bar{P}') \max \{c(x) : x \in \bar{X}'\} \quad \bar{X}' \supseteq X \longrightarrow x^* \in \bar{X}' \end{array}$$

BRANCH & BOUND su TSP

Dato un grafo non completo vogliamo un CICLO HAMILTONIANO (tocca tutti i nodi una volta) di costo MINORE.

① Calcolo $\bar{z}$ solo una volta del nodo radice

② Calcolo l'albero di copertura di costo minimo a cui aggiungo il primo lato di costo minimo scartato
Il costo dell'albero di copertura corrisponde al nuovo $\bar{z}$

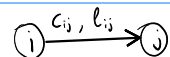


③ Prendo un nodo con $2+n$ lati nell'ALBERO (devono essere 2 per un ciclo) e faccio branching su $2+n$ casi del tipo $X_{ba}=\emptyset, X_{ca}=\emptyset, X_{da}=\emptyset, \dots$ e ripeto ②

$\bar{z} = \min \{ \bar{z}(P') : (P') \in Q' \}$, ovvero il minimo valore tra gli alberi dei nodi foglia non "TERMINATI"

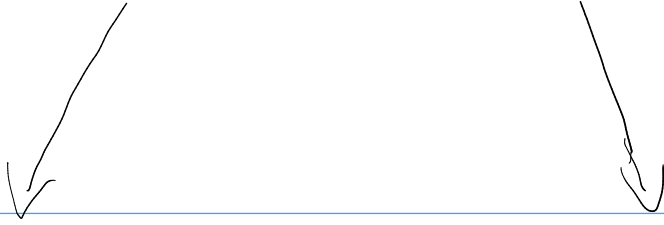
BRANCH & BOUND su CAMMINO MINIMO VINCOLATO

Cammino minimo tra s e t con lunghezza totale $L \leq 10$



① $\bar{z} = +\infty$

② Calcolo l'albero dei cammini minimi da s e considero solo gli archi del cammino fino a t
 $\bar{z}$ corrisponde al costo di quel cammino minimo



LUNGHEZZA AMMISSIBILE

$\bar{z}$ = costo dell'albero

LUNGHEZZA NON AMMISSIBILE

- ③ Partendo da s rimuovo il primo arco ($x_{si} = 0$), poi faccio un altro branch raggiungendo il primo e tagliando il secondo ($x_{si} = 1, x_{ij} = 0$) e così via fino a t . Poi riparto dal punto ②